21. Gün Staj langchain

21. Gün — LangChain Ekosistemi ve 4 Agent Katmanı

1. Önce “LangChain” kelimesindeki karışıklık

Metindeki ilk önemli nokta şu: **LangChain dendiğinde her zaman aynı şey kastedilmiyor.**

LangChain şu anlamlarda kullanılabiliyor:

```
LANGCHAIN
    │
    ├── Şirket
    │
    ├── Açık kaynak framework
    │
    ├── Belirli LangChain kütüphaneleri
    │
    └── Genel ekosistem
            │
            ├── LangChain
            ├── LangGraph
            ├── LangSmith
            └── LangSmith Deployment
```

O yüzden biri "LangChain kullanıyorum" dediğinde bunun **framework mü, şirket mi, yoksa genel ekosistem mi** olduğu bağlama göre değişebiliyor. Metin özellikle bu isimlendirme karmaşasına dikkat çekiyor.

2. LangChain neden ortaya çıktı?

LangChain 2022'nin sonlarında **Harrison Chase** tarafından açık kaynak bir kütüphane olarak geliştiriliyor.

İlk temel amaçlarından biri farklı LLM API'leriyle çalışmayı kolaylaştırmak.

Eskiden:

```
Uygulama
    │
    ├── OpenAI API kodu
```

```
    ├── Başka model API kodu
    ├── Prompt yönetimi
    ├── Zincirleme
    └── RAG işlemleri
```

gibi birçok şeyi kendimiz yönetirken LangChain bunların üzerinde bir **soyutlama katmanı** oluşturuyor.

Özellikle:

```
LLM çağrıları
Prompt Templates
Chains
RAG
Model bağlantıları
```

gibi yapıları standartlaştırmaya çalışıyor.

Zaten **Lang + Chain** isminin önemli taraflarından biri de LLM çağrılarını birbirine **zincirleme (chain)** fikri.

3. Sonra LangChain büyüyor

Metindeki tarihsel gelişimi çok basitleştirirsek:

```
2022
  ↓
LangChain
LLM çağrılarını kolaylaştırma

2023
  ↓
Çok hızlı büyüme
Chains / RAG / LCEL vb.

2024
  ↓
LangGraph
+
LangSmith

2025
  ↓
```

```
LangChain 1.0
Büyük ekosistem yenilemesi

Sonraki dönem
  ↓
Agent Engineering odağı
+
Deep Agents
```

Yani LangChain başlangıçta bugünkü gibi doğrudan **"agent engineering platformu"** merkezli değildi.

Zaman içerisinde agent sistemlerine çok daha fazla ağırlık veriliyor.

4. LangChain ekosistemindeki temel ürünler

Burayı net ayıralım.

LangChain

LLM uygulamaları ve agent geliştirmek için kullandığımız framework tarafı.

LangGraph

LLM/agent işlemlerini **graph ve state tabanlı workflow** şeklinde kontrol etmek için.

LangSmith

Çalışan LLM ve agent sistemlerini:

- izlemek,
- trace etmek,
- debug etmek,
- değerlendirmek

için kullanılıyor.

LangSmith Deployment

Agent'ları production ortamına **deploy etmek ve çalıştırmak için yönetilen altyapı**.

Metinde ayrıca bunun eski adının **LangGraph Platform** olduğu belirtiliyor.

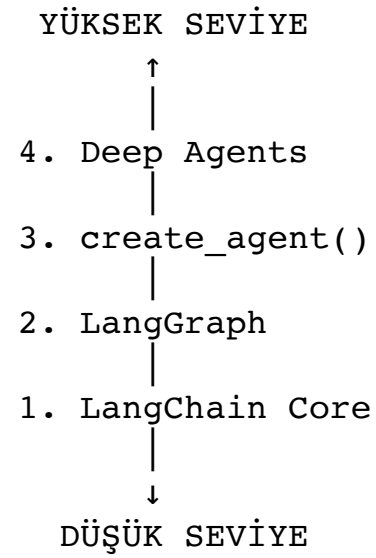
5. Dersin asıl önemli noktası: 4 katman

Buradan sonrası bizim için çok daha önemli.

Metin diyor ki:

LangChain ekosisteminde agent geliştirmenin dört farklı soyutlama seviyesi var.

Bunlar:



Aşağı indikçe: daha fazla kontrol + daha fazla kod.

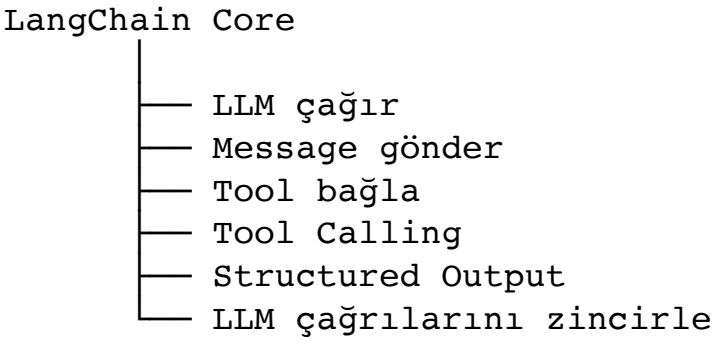
Yukarı çıktıkça: daha az uğraş + daha hazır yapı.

İşte bütün dersin kilit noktası bu.

Katman 1 — LangChain Core

En aşağıdaki temel yapı taşları.

Burada doğrudan LLM ile çalışıyoruz:



Örneğin önceki mesajımızdaki:

```
llm.invoke(...)
```

```
llm.stream(...)
llm.bind_tools(...)
```

gibi yapılar bu seviyeye ait.

Agent yapılabilir mi?

Evet.

Ama problem şu:

Agent döngüsünü büyük ölçüde kendimiz kurmak zorundayız.

```
User
 ↓
LLM
 ↓
Tool lazım mı?
 ↓
Tool Call
 ↓
Tool çalıştır
 ↓
Sonucu LLM'e ver
 ↓
Tekrar karar ver
 ↓
Final Answer
```

Bütün kontrol bizde olduğu için **çok esnek**, fakat daha zahmetli.

Metnin ifadesiyle bu seviyede güvenilir ve tekrarlanabilir agent yapmak için ciddi miktarda altyapı kurmak gerekiyor.

Katman 2 — LangGraph

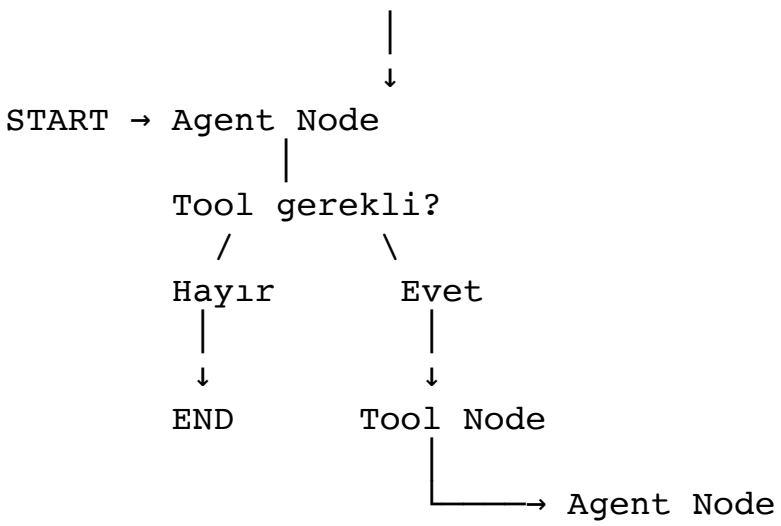
LangChain ekibi de diyor ki:

"Her seferinde bu agent workflow'unu elle yönetmeyelim."

Buradan **LangGraph** geliyor.

LangGraph'ta workflow'u graph şeklinde tanımlıyoruz:

```
                    STATE
```



Burada:

- Node
- Edge
- State
- Conditional Edge
- Graph

gibi yapılar kullanacağız.

Böylece LLM çağrıları ve tool işlemleri **kontrollü, tekrar edilebilir bir workflow** haline geliyor.

Ama hâlâ grafiği bizim tasarlamamız gerekiyor.

Yani:

LangGraph = Daha fazla workflow kontrolü, fakat hâlâ daha fazla geliştirme işi.

Bu kısmı zaten **Bölüm 2'de ayrıntılı öğreneceğiz.**

Katman 3 — `create_agent()`

Burada çok önemli bir fikir geliyor.

Çoğu agent aslında benzer döngüyü yapıyor:

```
LLM
 ↓
Tool
 ↓
LLM
 ↓
Tool
 ↓
```

```
LLM
 ↓
Final Answer
```

LangChain ekibi diyor ki:

"İnsanlar neden her seferinde LangGraph'ta bunu elle kursun?"

Bunun üzerine LangChain tarafında:

```
create_agent(...)
```

sunuluyor.

Biz kabaca:

```
agent = create_agent(
    model=model,
    tools=[...]
)
```

diyoruz.

Arka planda ise:

```
create_agent()
      ↓
LangGraph workflow
      ↓
LangChain Core
      ↓
LLM + Tools
```

çalışıyor.

İşte burası ilk bakışta kafa karıştırıyor.

`create_agent()` **LangChain içerisinde**, fakat arka planda **LangGraph kullanıyor**; LangGraph tarafı da temel model/tool bileşenleriyle çalışıyor.

OpenAI Agents SDK ile bağlantısı

Metin özellikle `create_agent()` yapısını OpenAI Agents SDK'ya benzetiyor.

Çünkü bizim istediğimiz çoğu zaman:

```
Modeli ver
+
Tools ver
+
MCP ver
+
Prompt ver
        ↓
      AGENT
        ↓
     Çalıştır
```

Bu yüzden **günlük agent geliştirme açısından en pratik katmanlardan biri** **`create_agent()`**.

Kursta da ticari Sidekick projesinde bu katmanın kullanılacağı söyleniyor.

Katman 4 — Deep Agents

En üst seviyeye geldik.

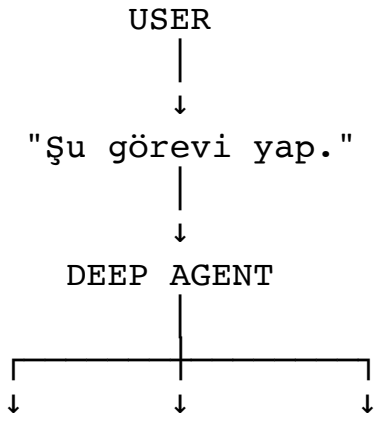
Burada artık agent'ın altyapısının büyük kısmı hazır geliyor.

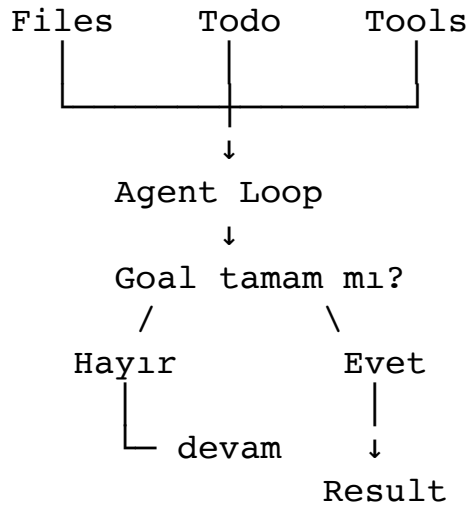
Deep Agents içerisinde metne göre örneğin:

- File System araçları
- Dosya okuma/yazma
- To-Do yönetimi
- Uzun süreli agent loop
- Goal-driven çalışma
- Yerleşik yetenekler

gibi şeyler bulunuyor.

Yani:





Bunun için metinde **"batteries included"**, yani kabaca **"her şey dahil"** yaklaşımı kullanılıyor.

Amaç:

Agent'ın loop'unu ve temel altyapısını tek tek kurmakla uğraşmadan ona hedef verip çalıştırmak.

Şimdi 4 katmanı yan yana koyalım

Katman	Ne yapıyoruz?	Kontrol	Kolaylık
LangChain Core	Her şeyi temel parçalarla kuruyoruz	Çok yüksek	Düşük
LangGraph	Workflow'u graph olarak kuruyoruz	Yüksek	Orta
`create_agent()`	Hazır agent workflow'u kullanıyoruz	Orta	Yüksek
Deep Agents	Hazır gelişmiş agent altyapısı	Daha az	Çok yüksek

En önemli nokta:

```
Deep Agents
     ↓ kullanır
create_agent / LangChain
     ↓
LangGraph
     ↓
LangChain Core
     ↓
LLM Provider
```

Bunu **araba kullanmaya** benzetebiliriz:

```
LangChain Core
→ Motor parçalarını kendin birleştir.

LangGraph
→ Arabayı kendin tasarla ama parçalar hazır.

create_agent()
→ Hazır arabayı al ve nasıl çalışacağını ayarla.

Deep Agents
→ Arabayı ver, hedefi söyle; birçok şey hazır gelsin.
```

Peki hangisini kullanacağız?

Burada "**Deep Agents en üstte, o zaman her zaman en iyisi**" diye düşünmemek gerekiyor.

İhtiyaca göre seçim yapıyoruz:

```
Sadece LLM + Tool mantığını öğrenmek/özelleştirmek
                  ↓
         LangChain Core


Workflow üzerinde ayrıntılı kontrol istiyorum
                  ↓
             LangGraph


Normal bir tool-using agent yapmak istiyorum
                  ↓
           create_agent()


Uzun ve karmaşık görev yapan,
hazır yetenekleri olan agent istiyorum
                  ↓
            Deep Agents
```

Özellikle bizim daha önce konuştuğumuz **Digital Twin 2.0** açısından LangGraph ilginç; çünkü orada:

```
User
```

```
 ↓
Orchestrator
 ├── RAG
 ├── Career
 └── General
 ↓
Evaluator
 ↓
Response
```

şeklinde **belirli ve kontrol etmek istediğimiz bir workflow** vardı.

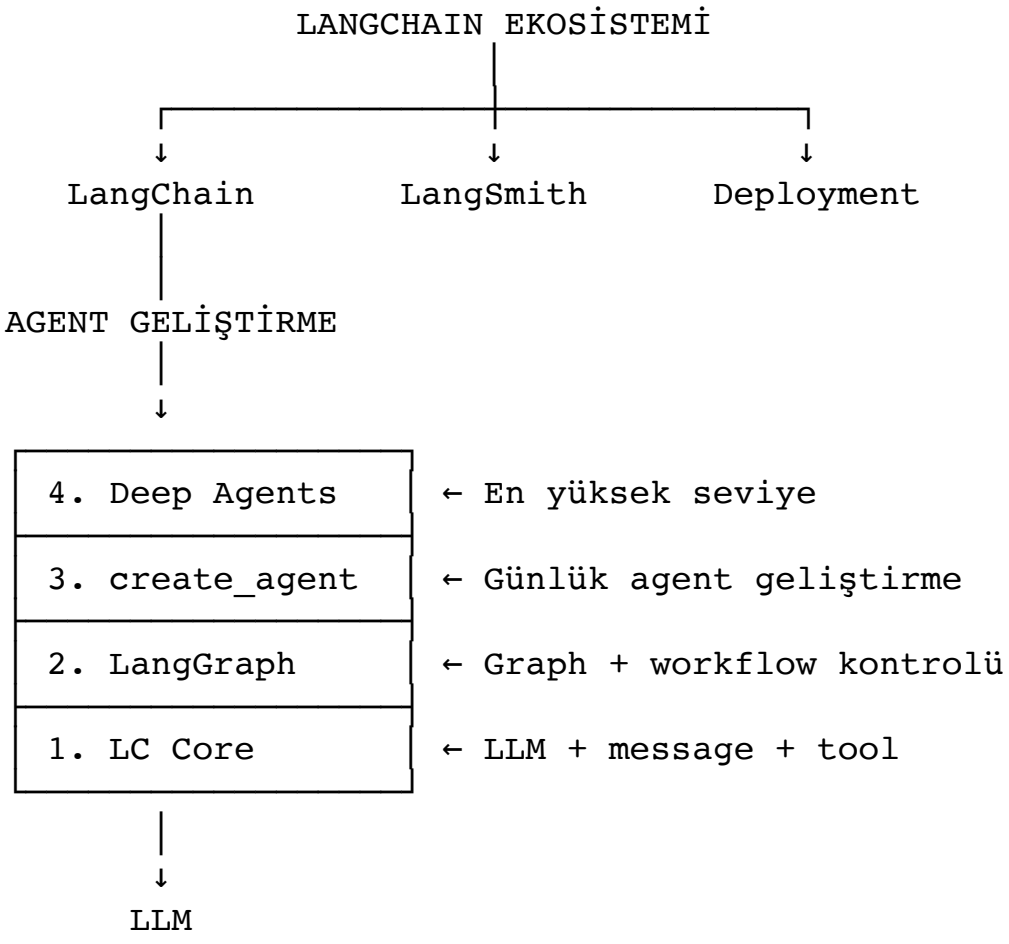
Ama çok daha standart:

```
LLM ↔ Tools ↔ LLM
```

agent'ı için her şeyi LangGraph ile elle çizmek gereksiz olabilir; `create_agent()` daha mantıklı olabilir.

Bu dersin bize anlatmak istediği şey

Metnin bütününü tek akışa indirirsek:



Bu dosyadan bugün aklımızda kalması gereken en önemli bilgi şu:

LangChain ekosisteminde agent geliştirmenin farklı soyutlama seviyeleri vardır: LangChain Core temel yapı taşlarını verir, LangGraph kontrollü workflow kurar, `create_agent()` yaygın agent döngüsünü hazır hale getirir, Deep Agents ise uzun ve karmaşık görevler için daha yüksek seviyeli, yerleşik yetenekleri olan bir agent altyapısı sunar.

21. Gün — Bölüm 1: LangChain Core ve Building Blocks

1. LangChain'de 4 soyutlama katmanı

LangChain ekosisteminde agent geliştirmek için 4 farklı seviye bulunuyor:

```
YÜKSEK SEVİYE
     ↑
4. Deep Agents
     │
3. create_agent()
     │
2. LangGraph
     │
1. LangChain Core
     ↓
DÜŞÜK SEVİYE
```

Mantık:

```
Aşağı indikçe
→ Daha fazla kontrol
→ Daha fazla manuel kod

Yukarı çıktıkça
→ Daha hazır yapı
→ Daha az kod
```

Katman 1 — LangChain Core

Temel yapı taşlarıdır:

```
LLM
Messages
Tools
Tool Calling
Streaming
Structured Output
```

Agent yapılabilir ama tool döngüsünü biz yönetiriz.

Katman 2 — LangGraph

```
State + Node + Edge + Graph
```

ile agent workflow'unu daha sağlam biçimde orkestre eder.

Katman 3 — `create_agent()`

Standart:

```
LLM ↔ Tool ↔ LLM
```

agent döngüsünü hazır verir.

Katman 4 — Deep Agents

Daha yüksek seviyelidir:

```
Planning
File System
To-Do
Sub-Agents
Long-running tasks
```

gibi özellikler hazır gelir.

2. LangChain Core ile kullandığımız temel importlar

Notebook'ta kabaca şu yapı kullanılıyor:

```
import os
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage,
SystemMessage, ToolMessage
from langchain_core.tools import tool

from pydantic import BaseModel, Field

load_dotenv(override=True)
```

Burada:

```
ChatOpenAI
→ Model bağlantısı

HumanMessage / SystemMessage / ToolMessage
→ Mesaj yapıları

@tool
→ Python fonksiyonunu Tool yapar

BaseModel / Field
→ Structured Output şeması
```

3. İlk LLM çağrısı — `ChatOpenAI`

Önce modeli oluşturuyoruz:

```
llm = ChatOpenAI(
    model="gpt-5.4-mini"
)
```

Bu model nesnesi üzerinden LLM ile iletişim kuracağız.

Mantık:

```
Python
 ↓
ChatOpenAI
 ↓
```

```
LLM Provider
 ↓
Model
```

`ChatOpenAI` sadece OpenAI modelleriyle sınırlı düşünülmemeli.

OpenAI-compatible API sağlayan başka servislerde de:

```
base_url=...
api_key=...
```

verilerek kullanılabilir.

Bu bizim Bulutistan kullanımımıza da oldukça benziyor.

4. `invoke()` — LangChain'in temel çalıştırma yöntemi

Modeli çalıştırmak için:

```
reply = llm.invoke(message)
```

kullanıyoruz.

Örneğin:

```
message = "In 1 sentence, what does it mean for an AI Agent
to be autonomous?"

reply = llm.invoke(message)

print(reply.content)
```

Akış:

```
Prompt
 ↓
llm.invoke()
 ↓
LLM
 ↓
```

```
AIMessage
 ↓
reply.content
```

En önemli fikir

`invoke()`:

Verilen input ile bu LangChain bileşenini çalıştır.

demektir.

Bu yüzden ileride de:

```
llm.invoke(...)
tool.invoke(...)
structured_llm.invoke(...)
graph.invoke(...)
agent.invoke(...)
```

göreceğiz.

5. `stream()` — cevabı parça parça alma

Tam cevabı beklemek yerine üretildikçe almak istersek:

```
for chunk in llm.stream(
    "Tell me a two line poem about autonomous agents"
):
    print(chunk.content, end="", flush=True)
```

kullanıyoruz.

Fark:

```
invoke()

Soru
 ↓
Bekle
 ↓
Tam cevap
```

```
stream()

Soru
 ↓
Parça
 ↓
Parça
 ↓
Parça
 ↓
Tamamlanır
```

Özellikle canlı UI'larda çok kullanışlıdır.

6. OpenAI-compatible başka provider kullanma

Notebook'ta OpenRouter örneği de gösteriliyor:

```
openrouter_llm = ChatOpenAI(
    model="anthropic/claude-haiku-4.5",
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENROUTER_API_KEY")
)
```

Sonra yine:

```
reply = openrouter_llm.invoke(
    "In one sentence, what is LangChain?"
)
```

Buradaki amaç:

Provider değişse bile LangChain tarafındaki kullanım şeklini mümkün olduğunca aynı tutmak.

```
ChatOpenAI
     │
     ├── OpenAI
     ├── OpenRouter
     └── başka OpenAI-compatible API
```

7. Messages — Dictionary yerine nesneler

Klasik API kullanımında:

```
messages = [
    {
        "role": "system",
        "content": "You are a terse assistant."
    },
    {
        "role": "user",
        "content": "What is the capital of France?"
    }
]
```

kullanabiliyoruz.

LangChain ise bunun için message nesneleri sunuyor:

```
messages = [
    SystemMessage(
        "You are a terse assistant who answers in exactly
five words."
    ),
    HumanMessage(
        "What is the capital of France?"
    )
]
```

Sonra:

```
llm.invoke(messages)
```

Temel message türleri

`SystemMessage`

Modelin davranışını belirler.

```
SystemMessage(
    "You are a financial assistant."
)
```

HumanMessage

Kullanıcının mesajıdır.

```
HumanMessage(
    "What is Amazon's share price?"
)
```

AIMessage

LLM'in cevabıdır.

ToolMessage

Tool çalıştıktan sonra sonucunu conversation'a eklemek için kullanılır.

Akış:

```
SystemMessage
      ↓
HumanMessage
      ↓
AIMessage
      ↓
ToolMessage
      ↓
AIMessage
```

Bu son `ToolMessage`, tool loop kısmında çok önemli olacak.

8. `@tool` — Python fonksiyonunu LangChain Tool'una dönüştürme

Notebook'ta örnek:

```
@tool
def get_share_price(symbol: str) -> float:
```

```
    """Return the current share price for a given ticker
symbol."""

    fake_prices = {
        "AAPL": 241.5,
        "GOOG": 168.2,
        "AMZN": 198.0
    }

    return fake_prices.get(symbol.upper(), 0.0)
```

Normalde bu bir Python fonksiyonudur.

Ama:

```
@tool
```

ekleyince LangChain bunu bir Tool nesnesine dönüştürür.

```
Python Function
      ↓
    @tool
      ↓
LangChain Tool
```

9. Tool'un metadata'sı

`@tool` sonrasında artık şunlara ulaşabiliyoruz:

```
get_share_price.name
```

Sonuç:

```
get_share_price
```

```
get_share_price.description
```

Buradaki açıklama docstring'den gelir:

```
"""Return the current share price for a given ticker
symbol."""
```

Argüman schema’sı da çıkarılır:

```
symbol
 ↓
string
```

Bu bilgiler LLM’in tool’u anlaması açısından önemlidir.

10. Docstring ve type hint neden önemli?

Şu fonksiyona bakalım:

```
@tool
def get_share_price(symbol: str) -> float:
    """Return the current share price for a given ticker
symbol."""
```

LLM açısından:

```
Tool name
→ get_share_price

Description
→ Return the current share price...

Argument
→ symbol

Argument type
→ string
```

bilgileri oluşur.

Yani:

```
Function name
+
Docstring
```

```
+
Type hints
       ↓
Tool Schema
       ↓
LLM
```

Bu nedenle agent tool'larında açıklamaları ve type hint'leri düzgün yazmak önemli.

11. Tool'u direkt çalıştırabiliriz

Tool olduktan sonra:

```
get_share_price.invoke({
    "symbol": "AAPL"
})
```

çalıştırabiliriz.

Örneğin sonuç:

```
241.5
```

Burada henüz LLM yok.

Direkt olarak tool'u çalıştırıyoruz.

12. `bind_tools()` — Tool'u modele tanıtma

Şimdi kritik noktaya geliyoruz.

Tool'u oluşturmak başka şey:

```
@tool
def get_share_price(...):
```

LLM'ye tanıtmak başka şey:

```
llm_with_tools = llm.bind_tools(
```

```
    [get_share_price]
)
```

Akış:

```
Python Function
      ↓
   @tool
      ↓
Tool oluştu
      ↓
bind_tools()
      ↓
LLM artık Tool'u biliyor
```

Çok önemli

```
bind_tools()
≠
tool'u çalıştırmak
```

Sadece modele:

Böyle bir aracın var.

diyoruz.

13. LLM tool’u kullanmak istediğinde ne döner?

Soruyoruz:

```
response = llm_with_tools.invoke(
    "What is the share price of Amazon?"
)
```

İlk sonuçta cevap doğrudan:

```
Amazon = $198
```

olmayabilir.

Notebook'ta:

```
print(response.content)
print(response.tool_calls)
```

yapılıyor.

Sonuç mantıksal olarak:

```
content:
""

tool_calls:
[
    {
        "name": "get_share_price",
        "args": {
            "symbol": "AMZN"
        }
    }
]
```

Yani LLM diyor ki:

Benim `get_share_price(symbol="AMZN")` tool'unu kullanmam gerekiyor.

Ama tool'u henüz çalıştırmadı.

14. Tool loop'u neden manuel yazıyoruz?

İşte LangChain Core'un düşük seviyeli kısmı burada ortaya çıkıyor.

İlk olarak conversation:

```
conversation = [
    HumanMessage(
        "What is the share price of Amazon?"
    )
]
```

Sonra:

```
ai_message = llm_with_tools.invoke(conversation)
```

Modelin tool request'ini conversation'a ekliyoruz:

```
conversation.append(ai_message)
```

15. Tool Call'ları geziyoruz

```
for call in ai_message.tool_calls:
```

ve hangi tool'un istendiğine bakıyoruz:

```
if call["name"] == "get_share_price":
```

Sonra gerçekten tool'u çalıştırıyoruz:

```
result = get_share_price.invoke(
    call["args"]
)
```

Burada artık gerçek Python fonksiyonu çalışıyor.

```
AMZN
 ↓
get_share_price
 ↓
198.0
```

16. `ToolMessage` neden ekliyoruz?

Tool sonucunu modele geri vermemiz gerekiyor.

```
conversation.append(
    ToolMessage(
        content=str(result),
        tool_call_id=call["id"]
    )
)
```

Buradaki:

```
tool_call_id=call["id"]
```

önemli.

Çünkü model:

Bu gelen sonuç benim hangi tool çağrıma ait?

diye anlayabiliyor.

Akış:

```
AIMessage
"get_share_price çağır"
        ↓
Tool çalışır
        ↓
198.0
        ↓
ToolMessage
"Bu sonuç o tool call'a ait."
```

17. LLM’yi ikinci kez çağırıyoruz

Artık conversation içerisinde:

```
HumanMessage
+
AI Tool Call
+
ToolMessage
```

var.

Sonra:

```
final = llm_with_tools.invoke(
    conversation
)
```

Bu kez LLM tool sonucunu görebiliyor.

Final cevap:

```
Amazon (AMZN) is at approximately $198.00 per share.
```

gibi oluyor.

18. İlk basit agent'ımız

Bütün sistem:

```
USER
"What is Amazon's share price?"
        ↓
LLM
        ↓
"get_share_price kullanmalıyım."
        ↓
Tool Call
symbol="AMZN"
        ↓
Python
        ↓
get_share_price.invoke()
        ↓
198.0
        ↓
ToolMessage
        ↓
LLM ikinci çağrı
        ↓
Final Answer
```

Bu aslında basit bir **AI Agent**.

Çünkü:

```
Hedef
 ↓
Karar
 ↓
Tool seçimi
```

```
 ↓
Eylem
 ↓
Sonuç
 ↓
Final cevap
```

var.

19. Ama neden buna “düşük seviye” diyoruz?

Çünkü:

```
for call in ...
if call["name"] == ...
tool.invoke(...)
ToolMessage(...)
llm.invoke(...)
```

gibi orchestration kodlarını elle yazdık.

Bir tane tool için kolay.

Ama düşün:

```
15 tool
Retry
Error handling
Memory
Evaluator
Human approval
Conditional routing
Multiple agents
```

oldu.

Kod hızla karmaşıklaşabilir.

İşte LangGraph’ın ortaya çıkma nedeni tam olarak bu.

```
LangChain Core
→ yapı taşları
```

```
LangGraph
→ bu yapı taşlarının orchestration'ı
```

20. Structured Output

Son bölümde LLM'in serbest metin yerine belirlediğimiz bir yapı döndürmesini sağlıyoruz.

Önce Pydantic şeması:

```
class Company(BaseModel):
    name: str = Field(
        description="The company name"
    )

    ticker: str = Field(
        description="The stock ticker symbol"
    )

    founded_year: int = Field(
        description="The year the company was founded"
    )
```

Bu LLM'e diyor ki:

```
Cevabında şu alanları istiyorum:

name → str
ticker → str
founded_year → int
```

21.with_structured_output()

Modeli bu şemaya bağlıyoruz:

```
structured_llm = llm.with_structured_output(
    Company
)
```

Sonra:

```
company = structured_llm.invoke(
    "Tell me about Amazon the technology company"
)
```

Sonuç:

```
Company(
    name="Amazon",
    ticker="AMZN",
    founded_year=1994
)
```

Yani elimizde düz metin değil, gerçek bir Pydantic nesnesi var.

22. Structured Output'un avantajı

Artık:

```
company.name
```

```
Amazon
```

```
company.ticker
```

```
AMZN
```

```
company.founded_year
```

```
1994
```

diyebiliriz.

Bu özellikle workflow'larda çok güçlü.

Örneğin:

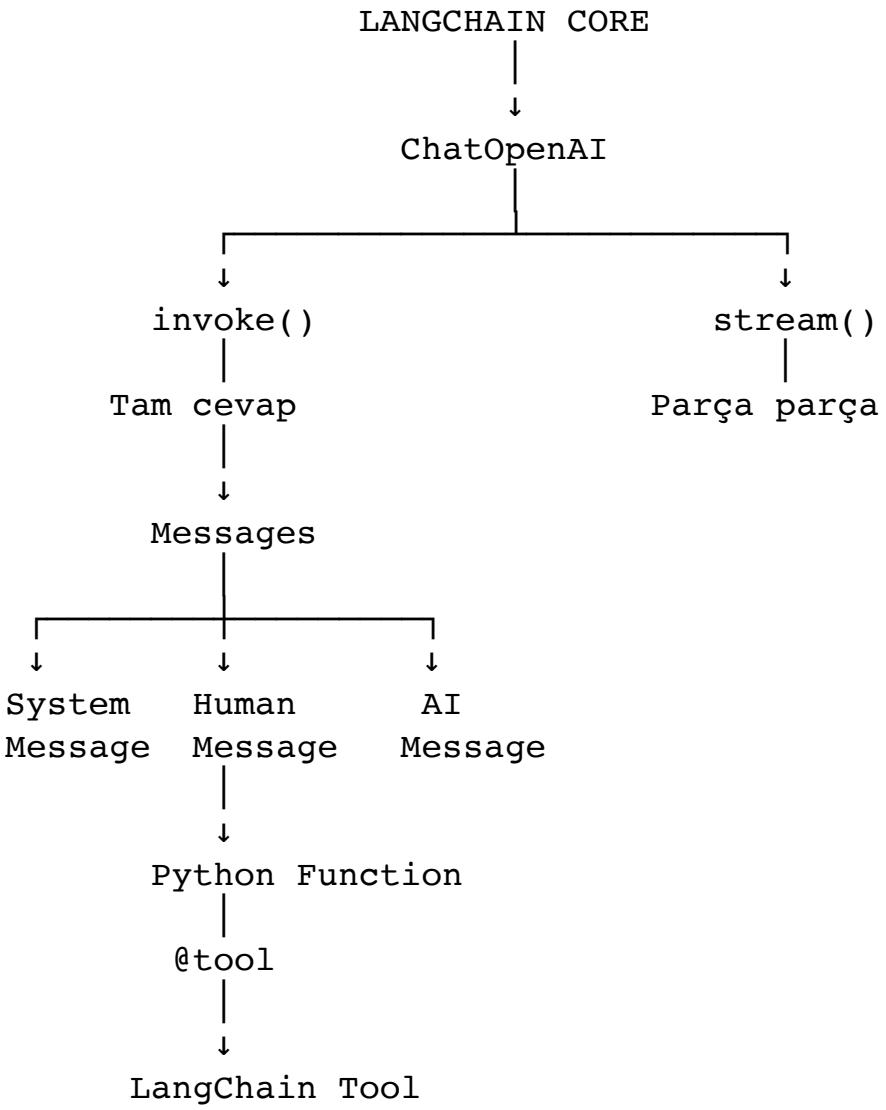
```
class Evaluation(BaseModel):
    approved: bool
    feedback: str
```

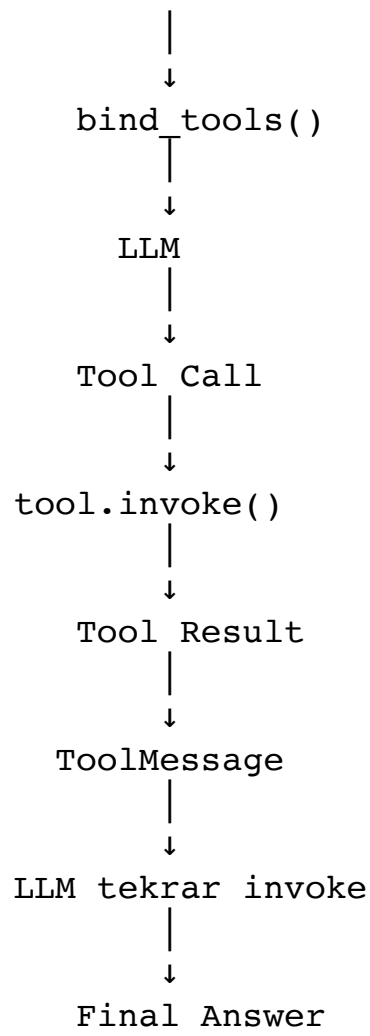
Sonra:

```
if evaluation.approved:
    ...
else:
    ...
```

yapabiliriz.

23. Notebook’taki bütün akış





Ayrı bir kolda:

```
LLM
 ↓
Pydantic Schema
 ↓
with_structured_output()
 ↓
invoke()
 ↓
Pydantic Object
```

24. LangChain Core'da bugün öğrendiğimiz temel komutlar

Kod	Görevi

`ChatOpenAI(...)`	LLM oluşturur
`llm.invoke(...)`	Modeli çalıştırır
`llm.stream(...)`	Cevabı stream eder
`SystemMessage(...)`	Sistem mesajı
`HumanMessage(...)`	Kullanıcı mesajı
`ToolMessage(...)`	Tool sonucunu modele verir
`@tool`	Python fonksiyonunu tool yapar
`tool.invoke(...)`	Tool'u çalıştırır
`llm.bind_tools(...)`	Tool'ları modele tanıtır
`response.tool_calls`	Modelin istediği tool çağrılarını verir
`with_structured_output(...)`	Çıktıyı belirli şemaya zorlar

25. Bölüm 1'in en kritik ayrımları

`@tool` vs `bind_tools()`

```
@tool
→ Fonksiyonu Tool yapar.

bind_tools()
→ Tool'u modele tanıtır.
```

`bind_tools()` vs `tool.invoke()`

```
bind_tools()
→ Model tool'un varlığını bilir.

tool.invoke()
→ Tool gerçekten çalışır.
```

İlk LLM çağrısı vs ikinci LLM çağrısı

```
1. LLM çağrısı
→ Hangi tool gerekli?

Tool çalışır.
```

```
2. LLM çağrısı
→ Tool sonucuna göre final cevap.
```

Normal Output vs Structured Output

```
Normal:
"Amazon was founded in 1994..."

Structured:
Company(
    name="Amazon",
    ticker="AMZN",
    founded_year=1994
)
```

Bölüm 1 — Tek cümlelik sonuç

LangChain Core, LLM çağrısı, message, tool, tool calling, streaming ve structured output gibi temel yapı taşlarını standartlaştırır; ancak agent'ın tool seçme → tool'u çalıştırma → sonucu LLM'e geri gönderme döngüsünü henüz bizim manuel olarak orkestre etmemiz gerekir.

21. Gün — LangGraph Orchestration Katmanı Özeti

1. LangGraph nedir?

LangGraph, bir workflow'daki işlemleri **graph yapısıyla tanımlayıp orkestre eden** bir framework'tür.

Temel amacı şu soruları açıkça belirlemektir:

- Hangi işlem yapılacak?
- Sonra hangi işlem çalışacak?
- Koşula göre hangi yol seçilecek?
- Hangi bilgiler node'lar arasında taşınacak?
- Workflow kaldığı yerden nasıl devam edecek?

LangGraph sadece LLM için değildir. Bir node; LLM çağrısı, tool kullanımı veya normal bir Python fonksiyonu olabilir.

```
Workflow
   ↓
LangGraph
```

```
   ↓
Node + Edge + State
```

2. Neden LangGraph kullandık?

LangChain Core'da agent loop'unu manuel yazıyorduk:

```
LLM
 ↓
Tool istedi mi?
 ↓
if / for
 ↓
Tool çalıştır
 ↓
ToolMessage
 ↓
LLM'yi tekrar çağır
```

Küçük sistemlerde bu yapılabilir.

Ama sisteme:

```
Birden fazla tool
Memory
Routing
Retry
Evaluator
Human Approval
Birden fazla LLM
```

eklendiğinde orchestration kodu karmaşıklaşır.

LangGraph burada:

Akışın nasıl çalışacağını `if/for/while` bloklarına gömmek yerine graph üzerinde açıkça tanımlamamızı sağlar.

Bu yüzden LangGraph'a **orchestration layer** diyoruz.

3. LangGraph'ın temel kavramları

Graph

Bütün workflow'dur.

```
START
  ↓
Node A
  ↓
Node B
  ↓
END
```

State

Workflow boyunca taşınan ortak bilgidir.

Örneğin:

```
{
    "messages": [...],
    "spanish": "...",
    "approved": True
}
```

State:

Sistemin o anki durumunun anlık görüntüsü gibi düşünülebilir.

Node

Bir işi gerçekleştiren Python fonksiyonudur.

```
def chatbot(state):
    ...
```

veya:

```
def translator(state):
    ...
```

Genel mantık:

```
State al
 ↓
İş yap
 ↓
Yeni state update'i döndür
```

Edge

Node’lar arasındaki geçiştir.

```
Node A
  ↓
Node B
```

A bittikten sonra B’nin çalışacağını söyler.

Conditional Edge

Bir koşula göre rota belirler.

Örneğin:

```
Chatbot
   ↓
Tool gerekli mi?
 /             \
Evet           Hayır
 ↓               ↓
Tools            END
```

Agent sistemlerinde en önemli yapılardan biridir.

4. State neden immutable mantıkla çalışıyor?

Node’un görevi eski state’i yerinde değiştirmek değil, **yeni bir update/state bilgisi döndürmektir**.

Örneğin:

```
return {
    "count": state["count"] + 1
```

```
}
```

Mantık:

```
State 1
  ↓
Node
  ↓
State 2
```

Bu yaklaşım özellikle paralel node'ların ve replay/checkpoint özelliklerinin daha düzenli çalışmasını sağlar.

5. Reducer nedir?

Birden fazla node state'e yeni veri döndürdüğünde:

```
Node A update ─┐
Node B update ─┼→ ?
Node C update ─┘
```

bunların mevcut state ile **nasıl birleştirileceğini** reducer belirler.

```
Mevcut State
+
Yeni Update
 ↓
Reducer
 ↓
Yeni State
```

add_messages

Mesaj listesinde kullanılan hazır reducer:

```
class State(TypedDict):
    messages: Annotated[list, add_messages]
```

Bu:

Yeni mesaj geldiğinde eski mesajların üzerine yazma, sona ekle.

demektir.

Örneğin:

```
HumanMessage("Merhaba")
+
AIMessage("Merhaba!")
```

sonuç:

```
[
 HumanMessage("Merhaba"),
 AIMessage("Merhaba!")
]
```

Conversation history böyle korunur.

6. LangGraph oluşturmanın 5 temel adımı

Burası en önemli pratik akışlardan biri.

1. State'i tanımla

```
class State(TypedDict):
    messages: Annotated[list, add_messages]
```

2. Builder oluştur

```
builder = StateGraph(State)
```

3. Node ekle

```
builder.add_node("chatbot", chatbot)
```

4. Edge ekle

```
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)
```

5. Compile et

```
graph = builder.compile()
```

Sonra:

```
graph.invoke(...)
```

ile çalıştırılır.

7. LangGraph yalnızca LLM değildir

İlk örnekte LLM kullanılmadan basit bir Python node'u oluşturuldu.

```
START
 ↓
Silly Node
 ↓
END
```

Bu örneğin amacı:

LangGraph'ın temel olarak bir orchestration framework olduğunu göstermek.

Sonra aynı node mantığı LLM ile kullanıldı:

```
START
 ↓
Chatbot Node
 ↓
END
```

Chatbot node:

```
State.messages
 ↓
LLM.invoke()
 ↓
AIMessage
 ↓
State
```

8. ToolNode ile Agent Loop

LangChain Core'da tool çağrısını biz elle yürütüyorduk.

LangGraph tarafında bunun için:

`ToolNode`

kullanıldı.

Görevi:

LLM'in istediği tool çağrılarını gerçekten çalıştırmak ve sonuçları state'e geri vermek.

Tool'lar yine:

```
llm_with_tools = llm.bind_tools(tools)
```

ile LLM'e tanıtılır.

Ama execution graph tarafından yönetilir.

9. Agent Graph

Temel agent döngüsü:

```
START
  ↓
CHATBOT
  ↓
Tool gerekiyor mu?
 /              \
EVET            HAYIR
 ↓                ↓
TOOLS             END
 ↓
CHATBOT
```

Burada:

```
Chatbot
→ Tool Call üretebilir

tools_condition
→ Tool gerekli mi diye karar verir

ToolNode
→ Tool'u gerçekten çalıştırır

Tool sonucu
→ Chatbot'a geri gider
```

Bu döngü ihtiyaç oldukça tekrar edebilir.

10. LangChain Core vs LangGraph

LangChain Core

```
LLM
 ↓
if
 ↓
for
 ↓
tool.invoke
 ↓
ToolMessage
 ↓
LLM.invoke
```

Orchestration'ı biz yazıyoruz.

LangGraph

```
Chatbot
   ↕
Tools
   ↓
END
```

Biz yalnızca:

```
Node'ları
+
Edge'leri
+
Koşulları
```

tanımlıyoruz.

Çalışma sırasını LangGraph yönetiyor.

11. Translator Node ile workflow büyütüldü

State'e yeni alan eklendi:

```
class State(TypedDict):
    messages: Annotated[list, add_messages]
    spanish: str
```

Translator:

```
Chatbot'un son cevabı
 ↓
Translator Node
 ↓
LLM
 ↓
Spanish Output
 ↓
state["spanish"]
```

Graph artık:

```
START
  ↓
CHATBOT
  ↓
Tool?
 /   \
Yes   No
 ↓     │
TOOLS  │
```

```
 ↓      |
CHATBOT
   ↓
TRANSLATOR
   ↓
END
```

haline geldi.

Bu, LangGraph'ın birden fazla LLM işlemini de aynı workflow'da koordine edebildiğini gösteriyor.

12. LangSmith

LangSmith, workflow'u çalıştırmaz.

Onun görevi:

Sistemin nasıl çalıştığını gözlemlemektir.

Görebildiğimiz bilgiler:

```
LLM çağrıları
Tool çağrıları
Input / Output
Token kullanımı
Latency
Maliyet
Execution Trace
```

Özet:

```
LangGraph
→ Workflow'u çalıştırır.

LangSmith
→ Workflow'un ne yaptığını gösterir.
```

13. Super-step

Bir `graph.invoke()` çalışması kendi içinde birden fazla execution aşamasından oluşabilir.

Bu aşamalara **super-step** denir.

Örneğin:

```
graph.invoke()

Super-step 1
→ Chatbot

Super-step 2
→ Tools

Super-step 3
→ Chatbot

Super-step 4
→ Translator
```

Dolayısıyla:

```
graph.invoke()
≠
tek super-step
```

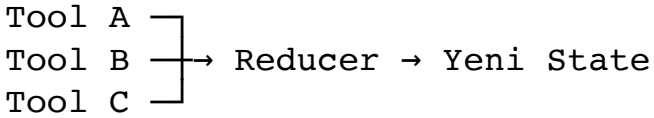
`graph.invoke()` tam bir graph run'dır.

Super-step ise onun içindeki bir çalışma aşamasıdır.

Paralel çalışan node'lar aynı super-step içinde olabilir.

14. Reducer ve Super-step ilişkisi

Aynı super-step içinde paralel node'lardan farklı update'ler gelebilir:



Reducer bu update'leri birleştirir.

Yani reducer:

Aynı execution sırasında state update'lerinin nasıl birleşeceğini yönetir.

15. Problem: Farklı `invoke()` çağrıları arasında memory yok

Örneğin:

```
Invoke 1:
"My name is Ed."

Invoke bitti.
```

Sonra:

```
Invoke 2:
"What is my name?"
```

Default durumda ikinci çağrı ilkini otomatik hatırlamaz.

Çünkü yeni bir graph run başlamıştır.

Bunu çözmek için:

Checkpointer

kullanılır.

16. Checkpointer

Checkpointer graph state'inin snapshot'larını saklar.

```
Super-step
 ↓
State
 ↓
Checkpoint

Super-step
 ↓
State
 ↓
Checkpoint
```

Böylece aynı workflow daha sonra kaldığı state'ten devam edebilir.

17. MemorySaver

İlk olarak:

```
memory = MemorySaver()
```

kullanıldı.

Graph:

```
graph = builder.compile(
    checkpointer=memory
)
```

şeklinde derlendi.

Böylece state checkpoint'leri memory'de tutuldu.

18. `thread_id`

Farklı konuşmaların checkpoint'lerini birbirinden ayırmak için:

```
config = {
    "configurable": {
        "thread_id": "conversation-1"
    }
}
```

kullanılır.

Mantık:

```
conversation-1
→ Kullanıcı A

conversation-2
→ Kullanıcı B
```

```
conversation-3
→ Kullanıcı C
```

Aynı `thread_id` kullanılırsa LangGraph önceki state'i yükleyebilir.

19. Memory örneği

İlk çağrı:

```
User:
My name is Ed.
```

aynı thread:

```
conversation-1
```

altında kaydedilir.

Sonra:

```
User:
What is my name?
```

LangGraph:

```
Checkpoint'i yükler
 ↓
Yeni mesajı ekler
 ↓
Graph'ı çalıştırır
```

ve:

```
Your name is Ed.
```

cevabını verebilir.

Node kodunu değiştirmedik.

Memory orchestration katmanında sağlandı.

20. MemorySaver ve SQLiteSaver

MemorySaver

```
RAM'de tutulur
Program kapanınca kaybolabilir
```

SQLiteSaver

```
SQLite dosyasına yazılır
Program kapansa bile saklanabilir
```

Örneğin:

```
SqliteSaver.from_conn_string("memory.db")
```

ile kalıcı checkpoint storage oluşturulur.

Bu yüzden:

```
MemorySaver
→ geçici memory

SQLiteSaver
→ persistent memory
```

21. Checkpoint sadece chat history değildir

Çok önemli.

Checkpoint şu tür bilgilerin tamamını tutabilir:

```
Messages
Tool sonuçları
Translation
Approval status
Agent outputs
Workflow variables
```

Yani:

Checkpoint, graph'ın o andaki tüm state'inin snapshot'ıdır.

Sadece sohbet geçmişi değildir.

22. `get_state()`

Mevcut state incelenebilir:

```
snapshot = graph.get_state(config)
```

Örneğin:

```
snapshot.values["messages"]
```

ile mevcut conversation içeriğini görebiliriz.

23. `get_state_history()`

Bütün checkpoint geçmişi:

```
graph.get_state_history(config)
```

ile alınabilir.

```
Checkpoint 1
Checkpoint 2
Checkpoint 3
Checkpoint 4
...
```

Böylece workflow'un hangi state'lerden geçtiği incelenebilir.

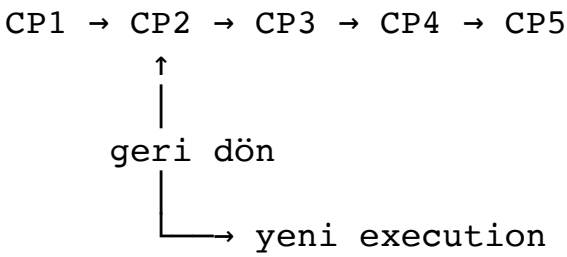
24. Time Travel / Replay

Her checkpoint'in:

```
checkpoint_id
```

değeri bulunur.

Eski bir checkpoint seçilerek workflow o noktadan tekrar başlatılabilir.



Bu özellikle uzun ve karmaşık agent workflow'larında çok güçlüdür.

25. Reducer vs Checkpointer

Bu ikisini kesinlikle karıştırmayalım.

```
REDUCER
→ Gelen state update'lerini nasıl birleştireceğim?
```

```
CHECKPOINTER
→ State'i nasıl kaydedip daha sonra geri yükleyeceğim?
```

Özet:

```
Reducer
→ merge

Checkpointer
→ save / restore
```

26. LangSmith vs Checkpointer

Bunlar da farklı görevler yapıyor.

```
LangSmith
→ Ne oldu?
→ Trace / debug / observe
```

```
Checkpointer
→ Nerede kaldık?
→ Memory / state history / recovery
```

27. Gradio UI

Son aşamada graph bir Gradio chat arayüzüne bağlandı.

Akış:

```
User Message
 ↓
Gradio
 ↓
graph.invoke()
 ↓
thread_id
 ↓
Checkpoint yükle
 ↓
Graph çalıştır
 ↓
Yeni checkpoint
 ↓
Response
 ↓
Gradio
```

Böylece kullanıcı basit bir chat ekranı görse de arka tarafta stateful LangGraph sistemi çalışıyor.

28. Final örnek

Kullanıcı önce:

```
My favorite fruit is banana.
```

diyor.

Sonra:

```
Send me a push notification
containing my favorite fruit.
```

diyor.

Sistem:

```
Checkpoint memory
 ↓
favorite fruit = banana
 ↓
Chatbot
 ↓
Tool Call
 ↓
Push Notification Tool
 ↓
"Your favorite fruit is banana."
```

şeklinde çalışıyor.

Ama perde arkasındaki gerçek mimari:

```
USER
 ↓
GRADIO
 ↓
graph.invoke()
 ↓
THREAD_ID
 ↓
CHECKPOINTER
 ↓
STATE
 ↓
CHATBOT LLM
 ↓
tools_condition
 ├──────────────┐
 ↓              ↓
TOOLS        TRANSLATOR
 ↓              ↓
CHATBOT         LLM
 └──────→       ↓
               END
```

```
  ↓
CHECKPOINT
```

Ve bütün bu execution:

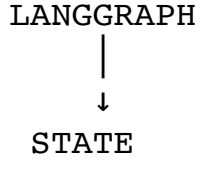
```
LANGSMITH
```

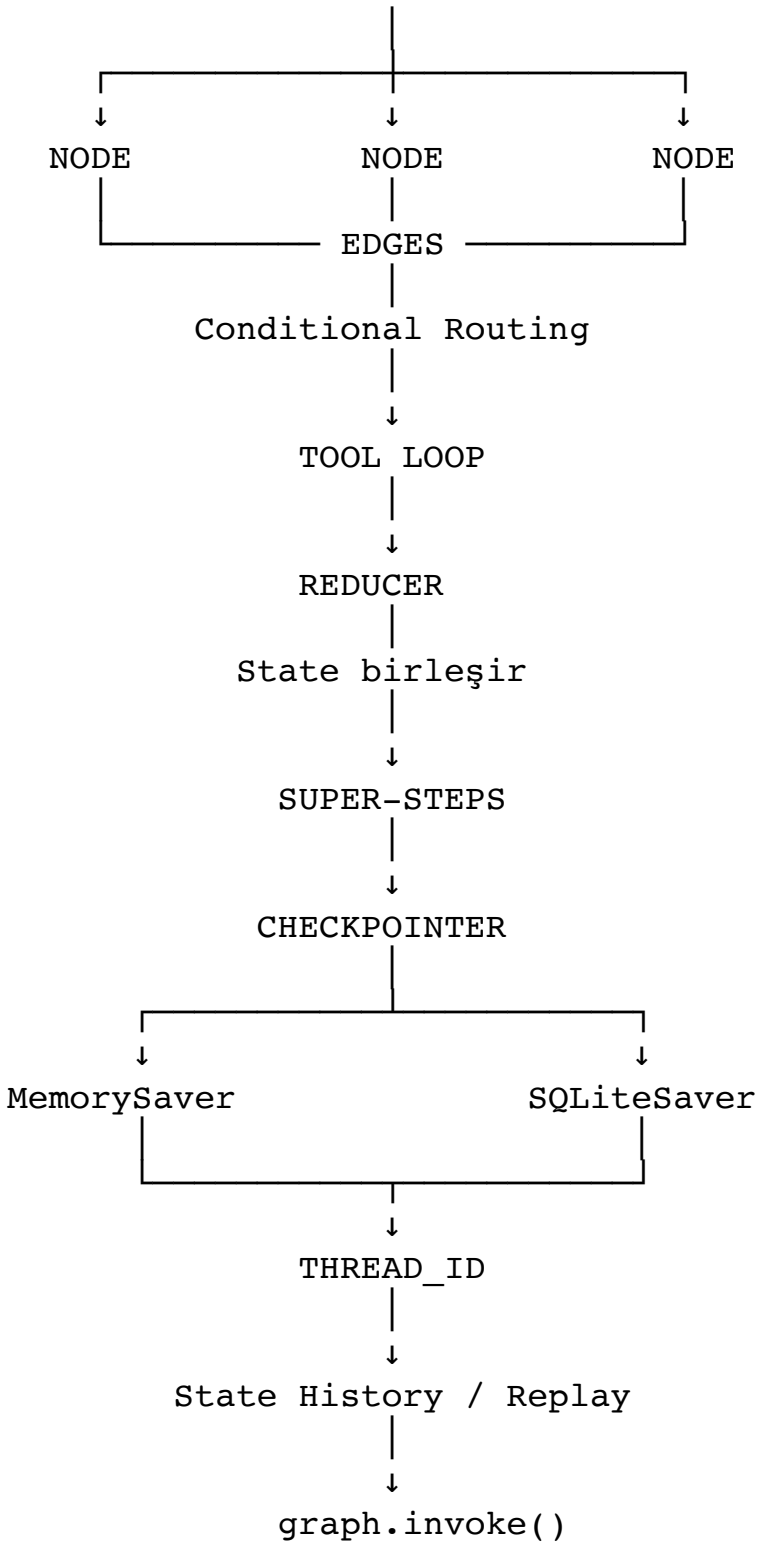
tarafından izlenebiliyor.

LangGraph için bilmemiz gereken ana kavramlar

Kavram	Görevi
Graph	Bütün workflow
State	Workflow boyunca taşınan veri
Node	Yapılan iş
Edge	Sonraki adımı belirler
Conditional Edge	Koşula göre rota seçer
Reducer	State update'lerini birleştirir
ToolNode	Tool çağrılarını çalıştırır
Super-step	Graph run içindeki execution aşaması
Checkpointer	State snapshot'larını saklar
thread_id	Conversation/state geçmişini ayırır
MemorySaver	RAM tabanlı checkpoint
SQLiteSaver	Kalıcı checkpoint
get_state()	Mevcut state'i gösterir
get_state_history()	State geçmişini gösterir
Replay / Time Travel	Eski checkpoint'ten devam eder
LangSmith	Trace, debug ve observability

LangGraph'ın tamamını tek akışta özetlersek





Dışarıdan da:

```
LANGSMITH
    ↓
Trace
Debug
Tokens
```

```
Latency
Cost
```

bütün sistemi gözlemler.

Tek cümlelik final tanım

LangGraph, agent workflow'unu State, Node ve Edge yapılarıyla açıkça tanımlayan; tool ve LLM çağrılarını koşullu döngülerle orkestre eden; Reducer ile state güncellemelerini birleştiren; Checkpointer ile memory, persistence ve replay sağlayan; LangSmith ile de tüm execution'ın gözlemlenebildiği stateful graph tabanlı orchestration framework'üdür.

1. `create_agent()` nedir?

`create_agent()` LangChain içindeki daha yüksek seviyeli agent oluşturma fonksiyonudur.

Temel kullanım:

```
agent = create_agent(
    model="openai:gpt-5.4-mini",
    system_prompt="You are a helpful assistant who answers
concisely."
)
```

Burada veriyoruz:

```
Model
+
System Prompt
+
İstersek Tools
+
İstersek Memory
+
İstersek Structured Output
+
İstersek Middleware
        ↓
    create_agent()
        ↓
```

```
      Agent
```

Sonra:

```
result = agent.invoke(...)
```

ile çalıştırıyoruz.

Yani LangChain Core'daki gibi `bind_tools`, `ToolMessage`, `if`, `for`, tekrar `invoke` gibi döngüyü tek tek yazmıyoruz.

2. Arka planda aslında ne oluyor?

En önemli nokta şu:

`create_agent()` aslında sihirli, bambaşka bir sistem değil.

Arka planda yine **LangGraph** kullanılıyor.

```
create_agent()
      ↓
Compiled LangGraph
      ↓
Model Node
      ↕
Tool Node
      ↓
Conditional Routing
```

Notebook'ta `agent.get_graph()` ile graph çizildiğinde bunu açıkça görüyoruz.

Yani dünkü:

```
START
 ↓
Chatbot
 ↓
Tool lazım mı?
 /        \
Evet      Hayır
 ↓          ↓
Tools      END
 ↓
```

```
Chatbot
```

graph'ını bugün tek fonksiyonla oluşturuyoruz.

İşte 2. ve 3. katman farkı:

```
LangGraph
→ Graph'ı sen tasarla.

create_agent()
→ Standart agent graph'ını ben hazırlayayım.
```

3. En basit Agent

Sadece model ve system prompt yeterli:

```
agent = create_agent(
    model="openai:gpt-5.4-mini",
    system_prompt="You are a helpful assistant."
)
```

Sonra:

```
result = agent.invoke({
    "messages": [
        {
            "role": "user",
            "content": "What is MCP?"
        }
    ]
})
```

Son cevaba:

```
result["messages"][-1].content
```

ile ulaşabiliyoruz.

Yani:

```
User
```

```
 ↓
create_agent
 ↓
LLM
 ↓
Response
```

Bu en basit kullanım.

4. `invoke()` ve `ainvoke()`

LangChain'de yine bildiğimiz:

```
invoke()
```

var.

Bunun async kardeşi:

```
ainvoke()
```

Örneğin:

```
result = await agent.ainvoke(...)
```

Fark:

```
invoke()
→ synchronous

ainvoke()
→ asynchronous
```

Agent'lar dış servis, browser, MCP ve network işlemlerini çok kullandığı için async kullanım özellikle faydalı.

5. Tool eklemek artık çok kolay

İki tool oluşturuyoruz:

```
@tool
def get_weather(city: str) -> str:
    ...

@tool
def get_population(city: str) -> str:
    ...
```

Sonra direkt agent'a veriyoruz:

```
agent = create_agent(
    model="openai:gpt-5.4-mini",
    tools=[
        get_weather,
        get_population
    ],
    system_prompt="You are a travel assistant."
)
```

Sonra:

```
"What is the weather and population of Rome?"
```

dediğimizde agent gerekli tool'ları kendi seçip çalıştırıyor.

6. Önceki gün ile farkı çok net

LangChain Core

Biz yazıyorduk:

```
bind_tools()
 ↓
LLM.invoke()
 ↓
tool_calls kontrol et
 ↓
for
 ↓
if
 ↓
```

```
tool.invoke()
 ↓
ToolMessage
 ↓
LLM.invoke()
```

LangGraph

Biz şunu kurduk:

```
Chatbot
 ↕
Tools
 ↓
END
```

`create_agent()`

Şimdi:

```
agent = create_agent(
    model=...,
    tools=[...]
)
```

diyoruz.

İşte katman yükseldikçe kod azalıyor.

7. Ama kontrolü tamamen kaybetmiyoruz

Burada önemli trade-off:

```
LangGraph
→ Çok fazla kontrol
→ Daha çok kod

create_agent()
→ Daha az manuel kontrol
→ Çok daha az kod
```

Standart:

`LLM ↔ Tools ↔ LLM`

agent'ları için `create_agent()` çok daha pratik.

Ama özel workflow gerekiyorsa hâlâ LangGraph'a inebiliriz.

8. Memory eklemek

Default durumda her `invoke()` yine yeni bir run.

Önce:

```
User:
I am planning a trip to London.

--- invoke bitti ---

User:
What is the weather where I am going?
```

Agent ikinci konuşmada Londra'yı unutabilir.

Bunu çözmek için yine dünkü mekanizma:

```
MemorySaver()
```

kullanılıyor.

Ama bu kez çok daha kısa:

```
agent = create_agent(
    model="openai:gpt-5.4-mini",
    tools=[get_weather],
    checkpointer=MemorySaver()
)
```

Sonra config:

```
config = {
    "configurable": {
        "thread_id": "trip-planning"
```

```
    }
}
```

Aynı `thread_id` ile:

```
Invoke 1
"I am planning a trip to London."

Invoke 2
"What is the weather where I am going?"
```

Agent Londra'yı hatırlıyor.

Burada çok önemli nokta:

`create_agent()` yeni memory sistemi icat etmiyor; altta yine LangGraph checkpointer kullanıyor.

9. Structured Output

Şimdi yine sevdiğimiz Pydantic yapısı geliyor.

Örneğin:

```
class CityReport(BaseModel):
    city: str
    weather: str
    population: str
```

Agent:

```
report_agent = create_agent(
    model="openai:gpt-5.4-mini",
    tools=[
        get_weather,
        get_population
    ],
    response_format=CityReport
)
```

Sonra:

```
"Give me a report on London."
```

diyoruz.

Agent:

1. gerekli tool'ları çalıştırıyor,
2. bilgileri topluyor,
3. sonuçları `CityReport` formatına sokuyor.

Sonuç:

```
CityReport(
    city="London",
    weather="Rainy, 14 degrees",
    population="8.9 million"
)
```

Bu sonuç:

```
result["structured_response"]
```

alanından alınabiliyor.

10. Structured Output neden güzel?

Çünkü tek agent run içinde:

```
Tool kullan
 ↓
Bilgi topla
 ↓
LLM reasoning
 ↓
Pydantic object
```

alabiliyoruz.

Yani ayrı ayrı:

```
Tool Calling
+
Result Parsing
+
```

`Schema Enforcement`

kodlarını elle yazmak gerekmiyor.

11. Middleware nedir?

Bölümün önemli yeni kavramlarından biri.

Middleware:

Agent loop'unun belirli noktalarına kendi kodumuzu araya sokmamızı sağlar.

Mesela:

```
Model çağrılmadan önce
Model çağrıldıktan sonra
Tool çağrılmadan önce
Tool çağrıldıktan sonra
```

gibi noktalara müdahale edebiliriz.

12. Tool Call Middleware örneği

Örnekte:

```
@wrap_tool_call
def log_tool_calls(request, handler):
    call = request.tool_call

    print(
        f"Middleware calling "
        f"{call['name']} with {call['args']}"
    )

    return handler(request)
```

Mantığı:

```
Agent tool çağırmak istiyor
        ↓
Middleware araya giriyor
        ↓
```

```
"Şu tool çağrılıyor" yazdır
        ↓
handler(request)
        ↓
Tool gerçekten çalışıyor
```

Yani middleware tool'u engellemiyor; önce bir işlem yapıp sonra normal execution'a devam ediyor.

13. Middleware neden güçlü?

Çünkü sadece loglama için kullanılmak zorunda değil.

Örneğin:

```
Logging
Guardrail
Tool izin kontrolü
Human approval
Retry
PII masking
Context management
Rate limiting
```

gibi işler yapılabilir.

Notebook'ta hazır middleware örneklerinden de bahsediliyor:

```
SummarizationMiddleware
PIIMiddleware
HumanInTheLoopMiddleware
Retry / call-limit benzeri yapılar
```

Yani middleware'i şöyle düşün:

Agent loop'unun içine takılabilen kontrol noktaları.

14. Guardrail ile benzerliği

Biz daha önce OpenAI Agents SDK tarafında guardrails konuşmuştuk.

Middleware tam olarak aynı şey değil ama benzer kullanım alanları var.

Örneğin:

```
Agent:
"example.com'a gideceğim"

Middleware:
"Bu domain blocklist'te mi?"

Evet
 ↓
Engelle

Hayır
 ↓
Tool çalışsın
```

Bu nedenle kullanıcıya verilen alıştırmada:

Belirli web sitelerine gitmesini middleware ile engelle.

deniyor.

15. Şimdi Browser Automation

Sonra konu browser tarafına geçiyor.

Ama önce LangChain değil, doğrudan:

Playwright

kullanılıyor.

Playwright, Microsoft'un browser automation framework'ü.

Kod ile:

```
Chrome aç
 ↓
Siteye git
 ↓
Tıkla
 ↓
Yaz
 ↓
Screenshot al
 ↓
```

```
Sayfayı oku
```

gibi işlemler yapılabilir.

16. Neden Node.js kuruluyor?

Playwright tarafındaki MCP server Node üzerinden çalışıyor.

Bu yüzden:

```
node --version
npx --version
```

kontrol ediliyor.

`npx` için güzel benzetme:

```
Python dünyası:
uvx / pip tabanlı araç çalıştırma

JavaScript dünyası:
npx
```

Yani npx bir npm paketini indirip doğrudan çalıştırabiliyor.

17. Önce Playwright tek başına test ediliyor

Örneğin:

```
Hacker News'e git
 ↓
Screenshot al
 ↓
playwright-check.png olarak kaydet
```

yapılıyor.

Burada henüz:

```
LLM yok
LangChain yok
```

```
LangGraph yok
Agent yok
```

Sadece:

```
Node
+
Playwright
+
Browser automation
```

test ediliyor.

Amaç:

Browser otomasyonu bilgisayarda gerçekten çalışıyor mu?

18. Sonra MCP devreye giriyor

Bölümün en güzel kısmı burası.

Biz Playwright için kendimiz tool yazmak isteseydik:

```
@tool
def navigate(...)

@tool
def screenshot(...)

@tool
def click(...)

@tool
def type_text(...)

...
```

diye birçok tool yazmamız gerekirdi.

Her birinde:

```
Tool name
Description
```

```
Arguments
Types
Playwright code
```

tanımlamak gerekirdi.

19. MCP burada ne sağlıyor?

Microsoft bunu bizim için zaten yapmış.

Playwright MCP Server

hazır browser tool'ları sunuyor.

Akış:

```
Agent
 ↓
MCP Client
 ↓
Playwright MCP Server
 ↓
Playwright
 ↓
Browser
```

MCP server bize browser kontrolü için hazır tool'lar sağlıyor.

Notebook'ta yaklaşık **23 tool** yüklendiği görülüyor.

Örneğin:

```
Navigate
Screenshot
Press key
Resize browser
Click
Read page
...
```

gibi.

20. Neden MCP çok değerli?

MCP olmasaydı:

```
23 farklı Python Tool
+
23 description
+
23 argument schema
+
23 Playwright implementasyonu
```

yazmamız gerekebilirdi.

MCP ile:

```
Microsoft bunları hazır sunuyor.
```

Biz sadece bağlanıyoruz.

İşte MCP'nin temel faydalarından biri bu:

Dış sistemlerin araçlarını agent'ların kullanabileceği standart bir protokol üzerinden sunmak.

21. MultiServerMCPClient

LangChain tarafında:

```
client = MultiServerMCPClient(...)
```

kullanılıyor.

Burada Playwright MCP server tanımlanıyor.

Transport:

```
stdio
```

olarak kullanılıyor.

Sonra:

```
browser_tools = await client.get_tools()
```

diyoruz.

Ve MCP server'ın sunduğu tool'lar LangChain Tool nesneleri gibi elimizde oluyor.

22. MCP tool'larını `create_agent()`a veriyoruz

Artık:

```
browser_agent = create_agent(
    model="openai:gpt-5.5",
    tools=browser_tools,
    system_prompt="""
    You are a web research assistant.
    Use the browser tools to complete the task.
    """
)
```

diyoruz.

Bak:

```
Normal Python @tool
veya
MCP'den gelen Tool
```

agent açısından benzer şekilde kullanılabiliyor.

Bu çok önemli.

23. Browser Agent nasıl çalışıyor?

Kullanıcı:

```
Go to Hacker News and tell me
the titles of the top three stories.
```

diyor.

Sonra async:

```
result = await browser_agent.ainvoke(...)
```

çalıştırılıyor.

Akış:

```
USER
 ↓
create_agent
 ↓
LLM
 ↓
Browser tool seç
 ↓
MCP
 ↓
Playwright
 ↓
Browser aç
 ↓
Hacker News
 ↓
Sayfayı incele
 ↓
Gerekirse başka tool
 ↓
LLM
 ↓
Goal tamam mı?
 /          \
Hayır       Evet
 ↓            ↓
Tool         Final Answer
```

İşte burada gerçekten **otonom agent hissi** oluşuyor.

24. Agent neden birden fazla kez browser kullanıyor?

Örnekte küçük model ilk denemede hemen doğru yolu bulamıyor.

Şunları yapıyor:

```
Navigate
 ↓
Screenshot
 ↓
Başka yaklaşım dene
 ↓
JavaScript / page inspection
 ↓
Tekrar navigate
 ↓
Bilgileri bul
```

Ama sonunda hedefe ulaşıyor.

Bu agent tanımımızla çok uyumlu:

LLM + tools + loop + goal

Yani:

```
Hedef tamamlanmadı
→ yeni tool kullan

Hedef tamamlandı
→ cevap ver
```

25. LangSmith burada yine devreye giriyor

Browser agent'ın neden bu kadar çok işlem yaptığını LangSmith üzerinden inceleyebiliyoruz.

Trace içinde:

```
LLM calls
Browser tool calls
Navigate
Screenshot
Page inspection
Latency
Token usage
Cost
```

görülebiliyor.

Böylece:

“Agent neden 30 saniye sürdü?”

sorusunu cevaplayabiliyoruz.

26. `create_agent()` + MCP birlikte neden güçlü?

Bütün sistemi çok az kodla kuruyoruz:

```
MCP Server
   ↓
browser_tools
   ↓
create_agent(
    model,
    tools,
    prompt
)
   ↓
ainvoke()
```

Ama arka planda:

```
LangGraph
 ↓
Agent Loop
 ↓
Tool Routing
 ↓
MCP
 ↓
Playwright
 ↓
Browser
 ↓
Repeated reasoning
```

çalışıyor.

Bu, üst soyutlama katmanlarının avantajını çok güzel gösteriyor.

27. Bu bölümde aslında ne yaptık?

Adım adım:

```
1. create_agent ile basit agent

2. invoke / ainvoke

3. @tool ile tool ekleme

4. Agent'ın arka planda LangGraph olduğunu görme

5. MemorySaver + thread_id

6. Structured Output

7. Middleware

8. Node.js kurulumu

9. Playwright browser automation

10. MCP Client

11. Playwright MCP Server

12. MCP tool'larını Agent'a verme

13. Browser kullanan autonomous agent

14. LangSmith ile trace/debug
```

28. Üç katmanı şimdi yan yana görelim

Katman 1 — LangChain Core

```
LLM
Tool
ToolCall
ToolMessage
Loop
```

hepsini daha manuel yönetiyoruz.

Katman 2 — LangGraph

```
State
Node
Edge
Conditional Edge
ToolNode
Checkpoint
```

ile workflow'u açıkça graph olarak kuruyoruz.

Katman 3 — `create_agent()`

```
Model
+
Tools
+
Prompt
+
Memory
+
Structured Output
+
Middleware
        ↓
    create_agent()
        ↓
Compiled LangGraph
```

Standart agent altyapısı otomatik kuruluyor.

29. `create_agent()` neden eğitmene göre "sweet spot"?

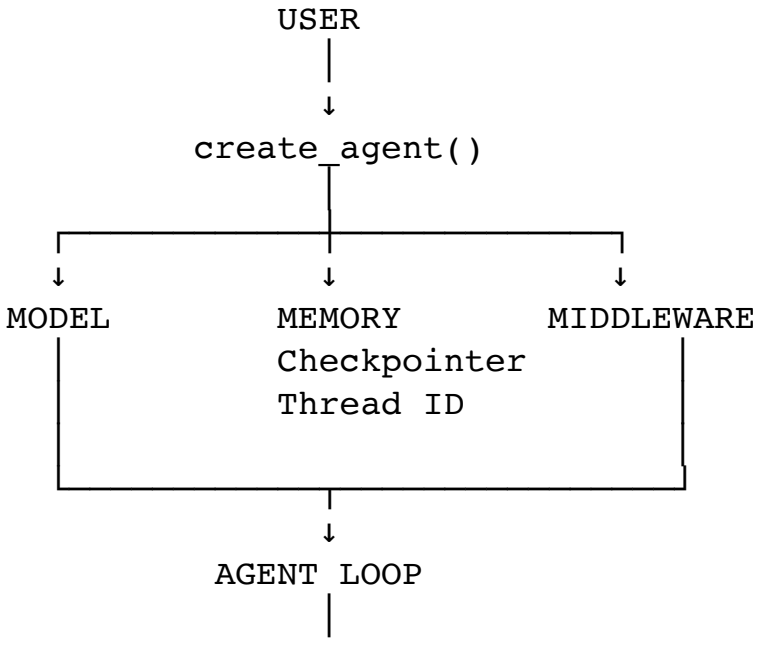
Çünkü:

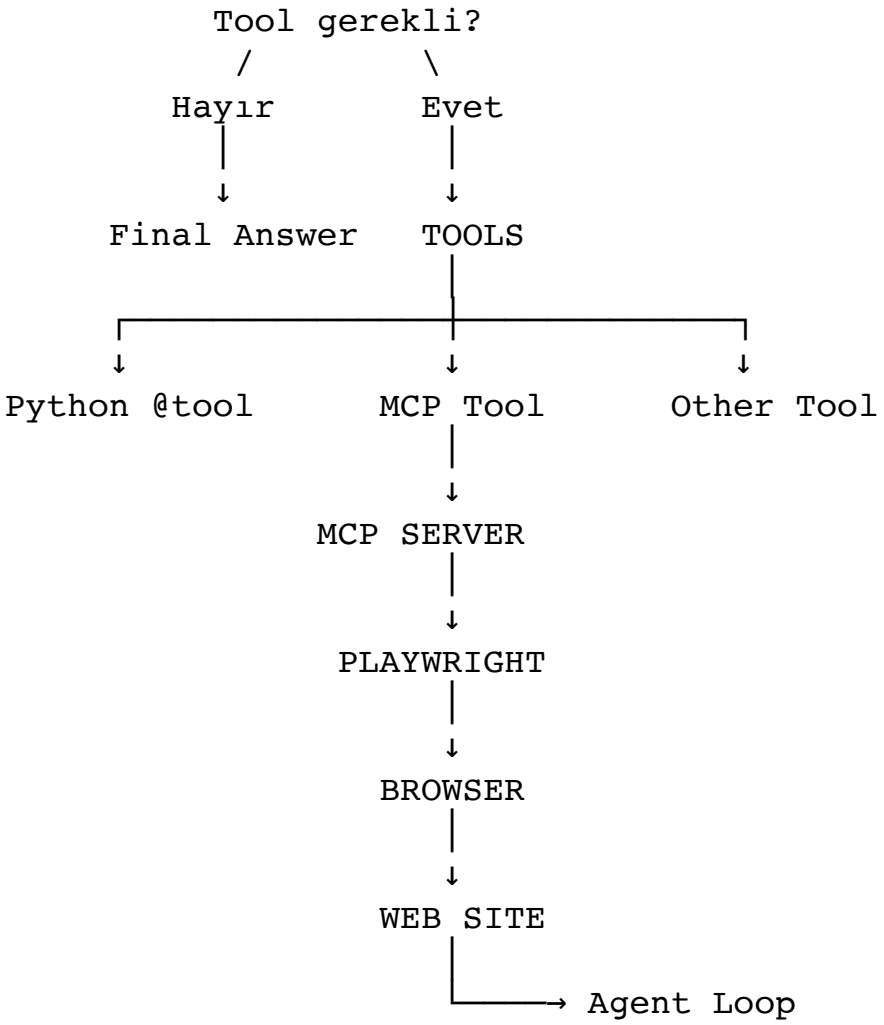
```
LangGraph kadar yorucu değil
+
LangChain Core kadar düşük seviye değil
+
OpenAI Agents SDK gibi rahat
+
Ama altında LangGraph'ın gücü var
```

Yani çoğu gerçek agent projesi için iyi bir denge:

```
Kolaylık
+
Kontrol
+
Tool Calling
+
Memory
+
MCP
+
Middleware
+
Structured Output
```

30. Bu bölümün bütün mimarisi





Ve altında görünmeyen ama çalışan:

```
create_agent()
      ↓
   LangGraph
      ↓
State + Node + Edge + Tool Loop
```

var.

En önemli 8 kavram

Kavram	Ne yapıyor?
`create_agent()`	Hazır agent loop oluşturur
`invoke()`	Senkron çalıştırır
`ainvoke()`	Asenkron çalıştırır
`tools=[...]`	Agent'a tool verir

`checkpointer`	Memory sağlar
`thread_id`	Conversation'ları ayırır
`response_format`	Structured output sağlar
`middleware`	Agent loop'una özel kontroller ekler
MCP	Harici tool sistemlerini bağlar
Playwright	Browser automation yapar

Tek cümlelik sonuç

`create_agent()`, LangGraph üzerinde hazır bir agent döngüsü kurarak model, tool, memory, structured output, middleware ve MCP gibi özellikleri çok daha az kodla kullanmamızı sağlar; böylece LangGraph'ın altyapı gücünü korurken esas olarak agent'ın çözmesi gereken iş problemine odaklanabiliriz.

21. Gün — Deep Agents Özeti

1. Deep Agents nedir?

Deep Agents, `create_agent()` yapısını alıp onun üstüne daha kapsamlı bir **agent harness** ekler.

Yani artık sadece:

```
Model + Tool + Prompt
```

ile çalışan bir agent yok.

Bunun yerine:

```
Model
+
Tools
+
File System
+
To-Do
+
Backend
+
Sub-Agents
```

```
+
Skills
+
Long-running loop
        ↓
    Deep Agent
```

vardır.

Deep Agent'a tek tek adımlar söylemek yerine daha büyük bir hedef veriyoruz:

```
"Bu işi araştır, planla, gerekli araçları kullan,
gerekirse alt ajanlara görev ver ve sonucu dosya olarak
üret."
```

2. `create_agent()` ile farkı

`create_agent()`

Daha çok standart agent loop'u:

```
User
 ↓
LLM
 ↓
Tool
 ↓
LLM
 ↓
Answer
```

Deep Agents

Daha kapsamlı:

```
Goal
 ↓
Plan
 ↓
To-Do
 ↓
Tools
```

```
 ↓
Files
 ↓
Sub-Agents
 ↓
Skills
 ↓
Artifact
```

Yani:

`create_agent()` agent döngüsünü hazırlar, Deep Agents ise agent'ın çalışacağı bütün ortamı hazırlar.

3. Harness ne demek?

Harness, agent'ın çevresindeki hazır çalışma altyapısıdır.

Örneğin:

```
To-Do Tool
File Tools
Search Tool
Sub-Agent Delegation
Skill Loading
Backend
Middleware
```

Deep Agent bunların çoğunu hazır getirir.

Bu yüzden biz yalnızca hedefi veririz; agent görevi nasıl yürüteceğine daha fazla kendisi karar verebilir.

4. Backend ve Sandbox

Deep Agent'a bir çalışma alanı verildi.

Örnekte:

```
sandbox/
```

klasörü kullanıldı.

`FilesystemBackend` ile agent bu klasörde:

```
Dosya okuyabilir
Dosya yazabilir
Dosya oluşturabilir
Çıktı saklayabilir
```

Örneğin laboratuvarda:

```
charging.md
fleet.md
fleet.pptx
```

dosyaları oluşturuldu.

Bunu agent'ın çalışma masası gibi düşünebiliriz:

```
Deep Agent
    ↓
FilesystemBackend
    ↓
sandbox/
```

5. İlk görev: EV Charging araştırması

İlk Deep Agent'a bir **web search tool** verildi.

Burada kullanılan:

```
Google Serper
```

idi.

Akış:

```
Deep Agent
 ↓
Serper Search
 ↓
Web Search
 ↓
```

```
Search Results
```

Agent’a verilen büyük hedef:

```
ABD’deki elektrikli araç şarj altyapısını araştır.
Public charging point sayısını bul.
İki büyük provider seç.
Sonucu charging.md dosyasına yaz.
```

6. Agent’ın görevi nasıl yürüttüğü

Biz tek tek:

```
Önce bunu ara
Sonra bunu ara
Sonra dosyaya yaz
```

demedik.

Agent kendi planını oluşturdu.

Gerçek tool sırası kabaca:

```
write_todos
 ↓
Search
 ↓
Search
 ↓
write_todos
 ↓
write_file
 ↓
write_todos
```

Yani:

```
Goal
 ↓
Plan yap
 ↓
Araştır
```

```
 ↓
Araştırmayı genişlet
 ↓
Sonuçları düzenle
 ↓
Dosyaya yaz
 ↓
Görevi tamamla
```

Bu, Deep Agent'ın built-in planning mantığını gösteriyor.

7. Built-in To-Do

Deep Agents'ın önemli özelliklerinden biri hazır To-Do sistemidir.

Agent görevi örneğin şöyle bölebilir:

```
1. Şarj ağı büyüklüğünü araştır
2. Büyük provider'ları bul
3. Verileri karşılaştır
4. Sonuçları yaz
5. Dosyaya kaydet
```

Bunu:

```
write_todos
```

gibi hazır tool'larla yönetir.

Biz ayrıca bir To-Do sistemi yazmadık.

8. Built-in File Tools

Agent ayrıca dosya araçlarını da hazır kullanabiliyor.

Örneğin:

```
read_file
write_file
```

Bu nedenle araştırmanın sonunda:

`charging.md`

dosyasını kendi oluşturabiliyor.

9. Deep Agents'ın altında yine LangGraph var

Deep Agent bambaşka bir altyapı değil.

Katmanlar şöyle:

```
Deep Agents
     ↓
create_agent()
     ↓
LangGraph
     ↓
LangChain Core
```

Yani Deep Agents daha üst seviyede ama altında yine daha önce öğrendiğimiz graph, tool loop ve model çağrıları bulunuyor.

10. Sub-Agent nedir?

Sonraki aşamada ana agent'a başka agent'lara görev devretme yeteneği verildi.

Örneğin:

```
Vehicle Researcher
```

adında bir sub-agent oluşturuldu.

Görevi:

```
Tek bir elektrikli aracı araştır
 ↓
Filo açısından önemli bilgileri bul
 ↓
Kısa sonuç döndür
```

11. Lead Agent + Sub-Agent yapısı

Ana agent'a görev:

```
Tesla Model Y ile Ford Mustang Mach-E'yi
100 araçlık filo için karşılaştır.
```

verildi.

Ana agent:

```
Tesla araştırması
 ↓
Vehicle Researcher

Ford araştırması
 ↓
Vehicle Researcher
```

şeklinde işi devretti.

Sonra sonuçları kendisi birleştirdi:

```
Tesla Result
+
Ford Result
 ↓
Comparison
 ↓
Recommendation
 ↓
fleet.md
```

Bu gerçek bir **manager-worker / delegation** yapısıdır.

12. `task` Tool

Ana agent'ın sub-agent'a görev vermesi için hazır:

```
task
```

tool'u kullanıldı.

Akış:

```
Main Agent
 ↓
Task Tool
 ↓
Sub-Agent
 ↓
Result
 ↓
Main Agent
```

Delegation mekanizmasını da ayrıca biz yazmadık; Deep Agents bunu hazır sağladı.

13. Skills nedir?

Skills, agent'a ek **bilgi, çalışma yöntemi veya stil kuralları** öğretmek için kullanılır.

Temel dosya:

```
SKILL.md
```

Örneğin PowerPoint skill'i agent'a:

```
Başlık kısa olsun
3 önemli bullet olsun
Recommendation ver
Şirket stilini kullan
```

gibi kuralları öğretiyor.

14. `SKILL.md` yapısı

Dosya iki parçadan oluşur.

Önce YAML frontmatter:

```
---
name: fleet-slide
```

```
description: Company slide style
---
```

Sonra Markdown talimatları:

```
# Slide Rules

- Short headline
- Three bullets
- Clear recommendation
- Follow company style
```

Yani:

```
SKILL.md
│
├── Metadata
│    ├── name
│    └── description
│
└── Detailed Instructions
```

15. Progressive Disclosure

Skills'in en önemli özelliği budur.

Agent başta bütün `SKILL.md` dosyasını context'e yüklemez.

İlk olarak yalnızca:

```
name
description
```

görür.

Eğer skill göreve uygunsa:

```
Skill gerekli
 ↓
SKILL.md oku
 ↓
Detayları context'e al
```

Bu yaklaşım:

Progressive Disclosure

olarak adlandırılır.

Avantajı:

```
Context küçülür
Token azalır
Gereksiz bilgi yüklenmez
Agent yalnızca ihtiyaç duyduğu skill'i açar
```

16. Skill ile Tool farkı

Bu bölümün en kritik ayrımlarından biri:

```
SKILL
→ İşin nasıl yapılacağını öğretir.

TOOL
→ İşi gerçekten yapar.
```

Örneğin PowerPoint:

```
Skill
→ Slayt nasıl görünmeli?

Tool
→ .pptx dosyasını gerçekten oluştur.
```

17. Neden her şeyi Skill yapmıyoruz?

Çünkü Tool Calling daha programatik ve daha güvenilir olabilir.

Tool:

```
Schema
Typed Arguments
Structured Call
```

kullanabilir.

Skill ise daha çok doğal dil talimatlarına dayanır.

Bu yüzden:

```
Kritik işlem
→ Tool

Stil / yöntem / rehber
→ Skill
```

ayrımı mantıklıdır.

18. PowerPoint örneği

İlk olarak elimizde:

```
fleet.md
```

vardı.

Sonra amaç:

```
fleet.md içindeki recommendation'dan
şirket stiline uygun PowerPoint oluştur.
```

Bunun için iki şey kullanıldı:

```
SKILL.md
+
create_slide Tool
```

19. `slide_kit.py`

Teknik PowerPoint üretim kodu:

```
slide_kit.py
```

dosyasındaydı.

Bu dosya:

```
Layout
Font
Logo
Brand colors
Placement
PowerPoint generation
```

gibi teknik işleri yönetiyordu.

Akış:

```
create_slide Tool
 ↓
slide_kit.py
 ↓
PowerPoint
```

20. Slide Maker Sub-Agent

Sunum işi için ayrı uzman agent oluşturuldu:

```
Slide Maker
```

Bu sub-agent'a:

```
create_slide Tool
+
Skill Directory
```

verildi.

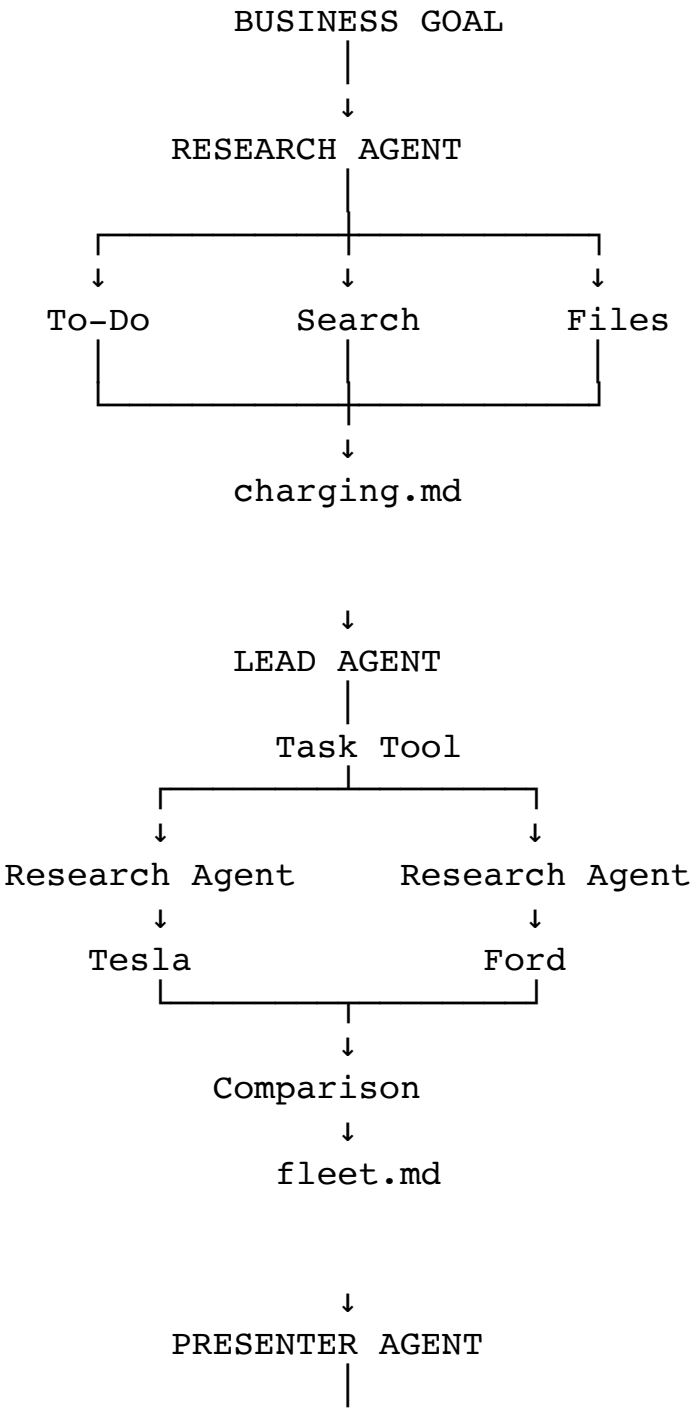
Görevi:

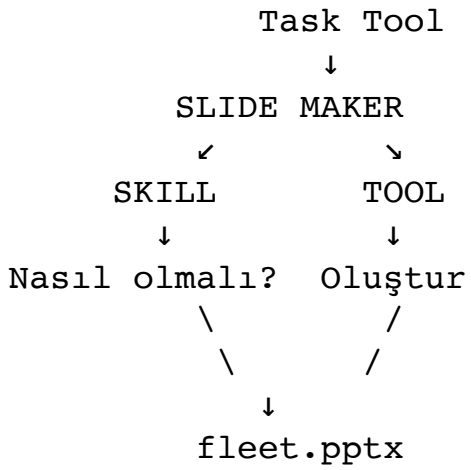
```
fleet.md oku
 ↓
Skill'i kullan
 ↓
```

```
Company style'ı öğren
 ↓
create_slide Tool
 ↓
fleet.pptx
```

21. Final Multi-Agent akışı

Bütün laboratuvarın gerçek hikâyesi:





22. Deep Agent içindeki roller

Bunu üç kavramla çok rahat hatırlayabiliriz:

Tool

İşi yapar.

Örneğin:

Search
Dosya yazma
PowerPoint oluşturma

Skill

İşin nasıl yapılacağını öğretir.

Örneğin:

Şirketin PowerPoint standardı

Sub-Agent

İşin belirli bir bölümünde uzmanlaşır.

Örneğin:

```
Vehicle Researcher
Slide Maker
```

Ve:

Deep Agent

```
Bütün bunları hedef etrafında organize eder.
```

23. LangSmith'in rolü

Deep Agents yüksek seviyeli ve "batteries included" olduğu için perde arkasında çok fazla işlem olabilir.

LangSmith ile:

```
Tool Calls
Sub-Agent Calls
Search Calls
To-Do
File Writes
Middleware
LLM Calls
```

izlenebilir.

Yani:

```
Deep Agents
→ Çalıştırır

LangSmith
→ İçeride ne yaptığını gösterir
```

24. Deep Agents ne zaman mantıklı?

Basit:

```
"Sıcaklık nedir?"
```

gibi bir görev için Deep Agent gereksiz olabilir.

Ama:

```
Araştır
Planla
Birden fazla kaynak kullan
Dosya oluştur
Alt ajanlara görev ver
Sonuçları birleştir
Sunum oluştur
```

gibi **uzun, çok adımlı ve artifact üreten görevlerde** çok uygundur.

25. Dört katmanı son kez yan yana koyalım

Katman	Ana fikir
LangChain Core	LLM, messages, tools gibi yapı taşları
LangGraph	State + node + edge ile orchestration
create_agent()	Hazır agent loop
Deep Agents	Hazır agent harness + Files + To-Do + Sub-Agent + Skills

Mantık:

```
Soyutlama ↑
Kolaylık ↑

Kontrol ↓
```

Bu bölümün tek büyük akışı

```
USER GOAL
   ↓
DEEP AGENT
   ↓
PLAN
   ↓
```

```
TO-DO
   ↓
SEARCH / TOOLS
   ↓
FILES
   ↓
SUB-AGENTS
   ↓
TASK DELEGATION
   ↓
RESULTS
   ↓
SKILL gerekli mi?
   ↓
SKILL.md
   ↓
TOOL
   ↓
FINAL ARTIFACT
.md / .pptx / ...
```

En önemli sonuç

Deep Agents, `create_agent()` üzerine planning, built-in To-Do, filesystem, backend, sub-agent delegation ve progressive disclosure ile çalışan Skills yapısını ekleyerek, uzun ve çok adımlı iş hedeflerini daha otonom biçimde tamamlayan yüksek seviyeli bir agent harness oluşturur.

Sidekick Projesi — Toparlanmış Not

1. Sidekick nedir?

Sidekick, **LangChain `create_agent()` kullanılarak geliştirilen kişisel bir AI iş arkadaşıdır.**

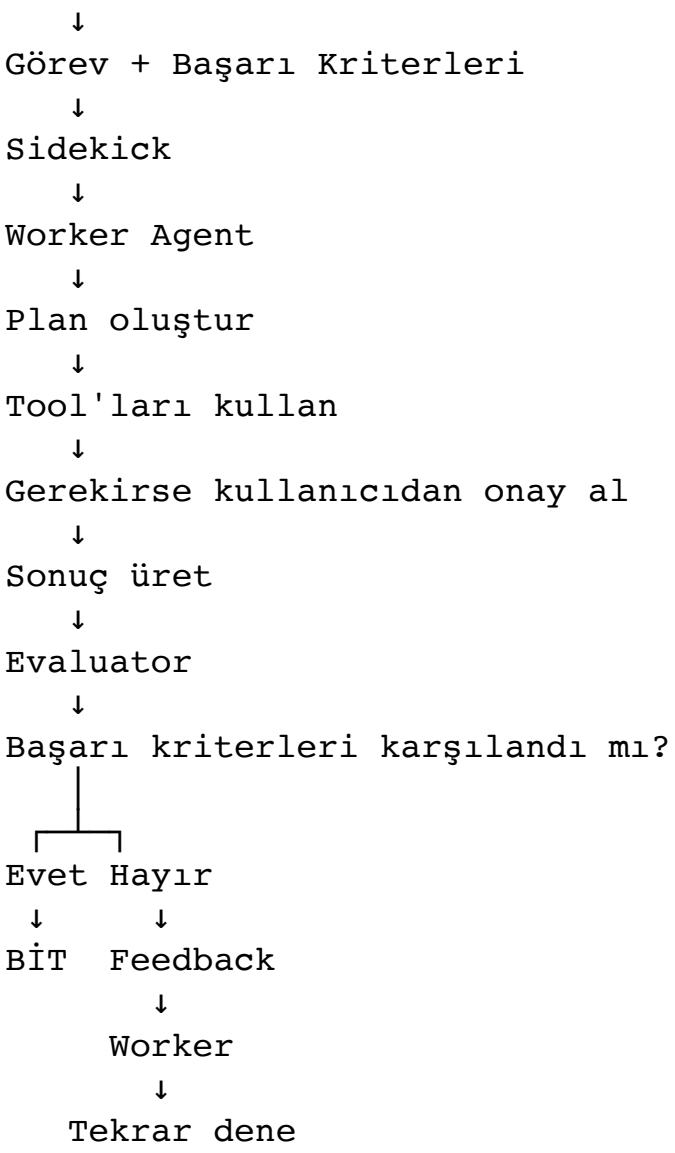
Kullanıcı Sidekick'e iki temel bilgi verir:

- **Task:** Yapılmasını istediği görev
- **Success Criteria:** Görevin hangi şartlarda başarılı sayılacağı

Sidekick görevi planlar, gerekli araçları kullanır, gerektiğinde kullanıcıdan izin ister ve sonunda yaptığı işi ayrı bir **Evaluator** ile kontrol ettirir.

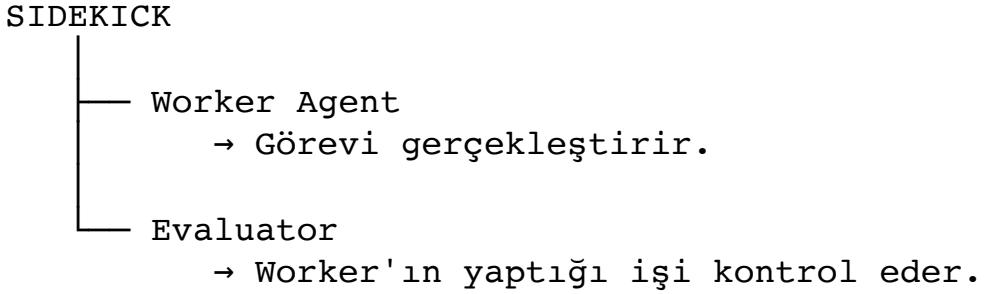
Temel mantık:

```
Kullanıcı
```



2. Sistemin iki temel parçası

Sidekick aslında iki ana AI bileşeninden oluşuyor:



Worker gerçek işi yaparken **Evaluator**, kullanıcının belirlediği başarı kriterlerinin gerçekten karşılanıp karşılanmadığını kontrol eder.

3. Worker Agent

Worker, `create_agent()` ile oluşturuldu.

İçerisinde:

```
Model
+
System Prompt
+
Tools
+
Memory
+
Middleware
```

bulunuyor.

Agent'a çok sayıda araç bağlandı. Final Sidekick örneğinde yaklaşık **41 tool** bulunuyor. Bunların önemli olanları:

```
Playwright
→ Web sitelerinde gezinme

Search
→ Web üzerinde araştırma

Wikipedia
→ Bilgi sorgulama

File Tools
→ Dosya okuma/yazma

Push Notification
→ Kullanıcıya bildirim gönderme

MCP Tools
→ Harici araçlara erişme
```

Böylece Sidekick yalnızca metin üreten bir chatbot değil, **araç kullanarak işlem gerçekleştirebilen bir agent** haline geliyor.

4. Todo List — Agent'ın planı

Sidekick'e **To-Do Middleware** ekleniyor.

Karmaşık bir görev geldiğinde agent önce kendi planını oluşturabiliyor.

Örneğin:

```
[ ] Uçuşları araştır
[ ] Fiyatları karşılaştır
[ ] En iyi 3 uçuşu seç
[ ] flights.md oluştur
[ ] En iyi seçimi bildir
```

Agent ilerledikçe bunları tamamlıyor.

Gradio arayüzünde kullanıcı da bu listeyi görebildiği için **agent'ın ne yaptığını canlı olarak takip edebiliyor.**

5. Middleware

Sidekick'in önemli özelliklerinden biri `create_agent()` etrafına çeşitli **middleware'ler** eklenmesi.

Todo Middleware

Agent'ın görevini planlamasını ve adımlarını takip etmesini sağlar.

PII Middleware

E-posta ve kredi kartı bilgileri gibi hassas bilgilerin korunmasına/maskelenmesine yardımcı olur.

Model Call Limit

Agent'ın kontrolsüz şekilde model çağırmasını önler.

Örnekte:

```
Maximum 30 model calls
```

sınırı kullanıldı.

Tool Error Middleware

Bir tool hata verdiğinde tüm agent'ın çökmesi yerine hata tekrar LLM'e iletiliyor:

```
Tool
```

```
 ↓
HATA
 ↓
LLM
 ↓
"Başka bir yöntem deneyeyim."
```

Bu da Sidekick'i daha dayanıklı hale getiriyor.

Human-in-the-Loop Middleware

Kritik bir işlem gerçekleştirilmeden önce **kullanıcı onayı** alınmasını sağlıyor.

6. Human-in-the-Loop

Sidekick tamamen başıboş şekilde hareket etmiyor.

Örneğin agent bir toplantı ayarlamak veya push notification göndermek istediğinde sistem işlemi durdurabiliyor:

```
Agent
 ↓
Kritik Tool Call
 ↓
INTERRUPT
 ↓
Kullanıcı
```

Kullanıcı üç farklı karar verebilir:

Approve: İşlemi onaylar ve tool çalışır.

Edit: Tool'a gönderilecek parametreleri değiştirir.

Reject: İşlem reddedilir ve agent gerekirse başka bir çözüm düşünür.

Daha sonra workflow **Resume** edilerek kaldığı yerden devam eder.

7. Memory

Sidekick'te konuşma durumunu koruyabilmek için bir **checkpointer** kullanılıyor.

Örnekte:

`InMemorySaver`

kullanılıyor.

Mantığı:

```
thread_id
   ↓
Conversation State
   ↓
Checkpoint
```

Böylece aynı thread içerisindeki agent çalışmaları ve konuşma durumu korunabiliyor.

8. Evaluator

Worker işi bitirdiğinde sistem doğrudan:

"Tamam, görev başarıyla tamamlandı."

demiyor.

Ayrı bir LLM olan **Evaluator** devreye giriyor.

Evaluator'a kabaca:

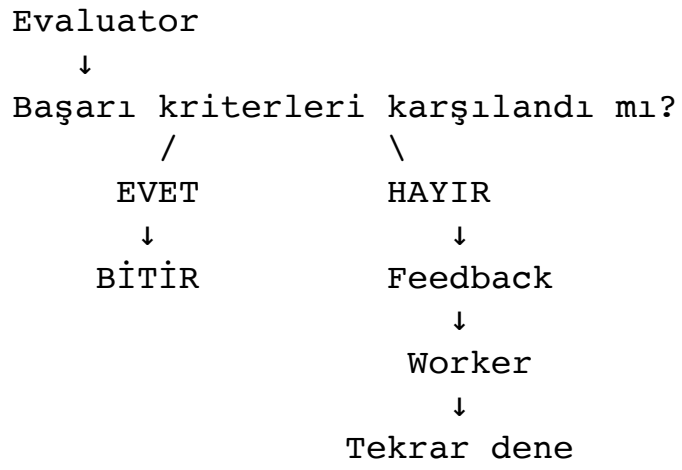
```
User Task
+
Success Criteria
+
Used Tools
+
Worker'ın Final Answer'ı
```

veriliyor.

Evaluator **Structured Output** döndürüyor ve görevin gerçekten başarılı olup olmadığını değerlendiriyor.

Mantık:

```
Worker
   ↓
Sonuç
   ↓
```



Bu yapı **Evaluator–Retry Loop** oluşturuyor.

9. Ana Senaryo — Uçuş Arama

Sidekick'in bütün özelliklerinin birleştiği asıl örnek uçuş arama görevi.

Kullanıcı Sidekick'e kabaca:

New York'tan Londra'ya yaklaşık bir ay sonra gidiş, bir hafta sonra dönüş olacak şekilde uçuşları araştır. Önce fiyatı, sonra seyahat süresini değerlendir. En iyi üç seçeneği `flights.md` dosyasına yaz ve en iyi seçimi bana bildir.

diyor.

Sidekick önce planlıyor:

```
1. Uçuşları araştır
2. Fiyatları karşılaştır
3. Süreleri karşılaştır
4. En iyi 3 uçuşu seç
5. flights.md oluştur
6. En iyi seçimi belirle
7. Push notification gönder
```

Sonra Playwright ile browser'ı kullanıp uçuşları araştırıyor.

Ardından seçenekleri karşılaştırıyor ve **File Tool** kullanarak:

```
sandbox/flights.md
```

dosyasını oluşturuyor.

Yani agent sadece:

"Bence şu uçuş iyi."

demiyor.

Gerçek bir dosya/artifact oluşturuyor.

10. Push Notification ve insan onayı

Sidekick en iyi uçuşu belirledikten sonra kullanıcıya bildirim göndermek istiyor.

Fakat push notification kritik tool olarak ayarlandığı için:

```
Sidekick
 ↓
Push göndermek istiyor
 ↓
Human-in-the-Loop
 ↓
INTERRUPT
 ↓
Kullanıcı onayı
 ↓
Approve
 ↓
Resume
 ↓
Push Notification
```

şeklinde ilerliyor.

11. Evaluator son kontrolü yapıyor

İşlem tamamlandıktan sonra Evaluator:

```
3 uçuş bulundu mu?
Fiyatlar verildi mi?
Süreler değerlendirildi mi?
flights.md oluşturuldu mu?
En iyi seçenek belirlendi mi?
Bildirim gönderildi mi?
```

gibi başarı kriterlerini kontrol ediyor.

Hepsi karşılandıysa:

```
SUCCESS
↓
Görev tamamlandı.
```

Eksik varsa Worker'a feedback veriliyor ve Sidekick yeniden çalışıyor.

12. Gradio Arayüzü

Son aşamada bütün sistem **Gradio UI** içerisine yerleştirildi.

Arayüzde temel olarak:

```
Task
Success Criteria
Chat
To-Do List
Human Approval
```

alanları bulunuyor.

Böylece kullanıcı yalnızca sonucu değil, agent'ın planını da görebiliyor:

```
✓ Search flights
✓ Compare options
✓ Write flights.md
→ Send notification
```

Bu da Sidekick'in yaptığı işlemleri daha **gözlemlenebilir ve şeffaf** hale getiriyor.

13. LangSmith

Arka planda gerçekleşen agent işlemleri **LangSmith** üzerinden takip ediliyor.

Burada:

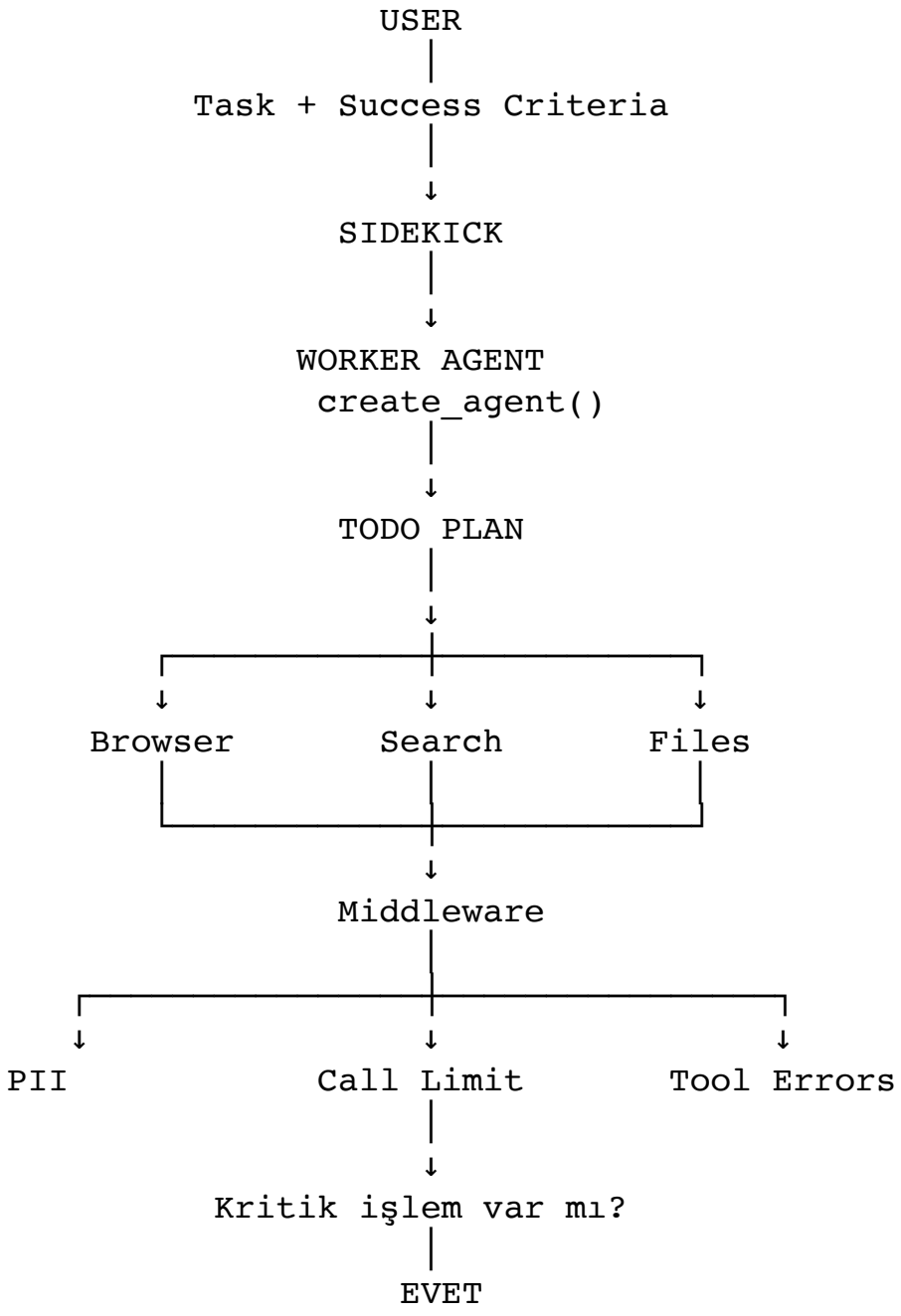
```
LLM Calls
Tool Calls
Middleware
Human Interrupt
```

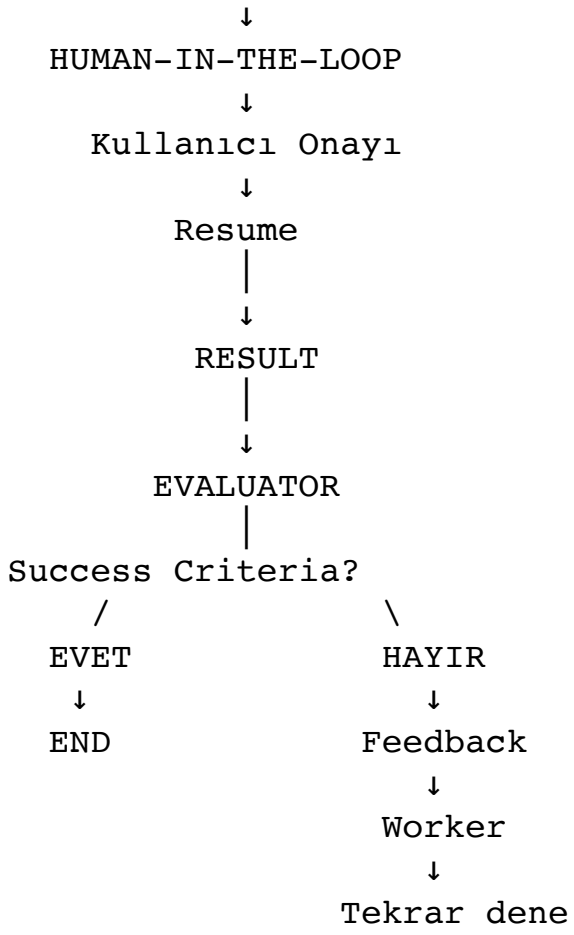
```
Resume
Evaluator
Latency
Tokens
```

gibi bilgiler incelenebiliyor.

Özellikle Human-in-the-Loop nedeniyle bir görev durdurulup daha sonra devam ettirildiğinde süreç birden fazla trace olarak görülebiliyor.

Sidekick'in Tam Akışı





En önemli çıkarım

Sidekick'in olayı yalnızca bir agent yapmak değil; `create_agent()` ile oluşturulan Worker'ı planning, tools, memory, middleware, human-in-the-loop ve ayrı bir Evaluator ile birleştirerek kontrollü ve hedef odaklı bir agent sistemi oluşturmaktır.